CONVOLUTIONAL NEURAL NETWORK

100000개 RGB픽셀 → 2개 점

200 픽셀

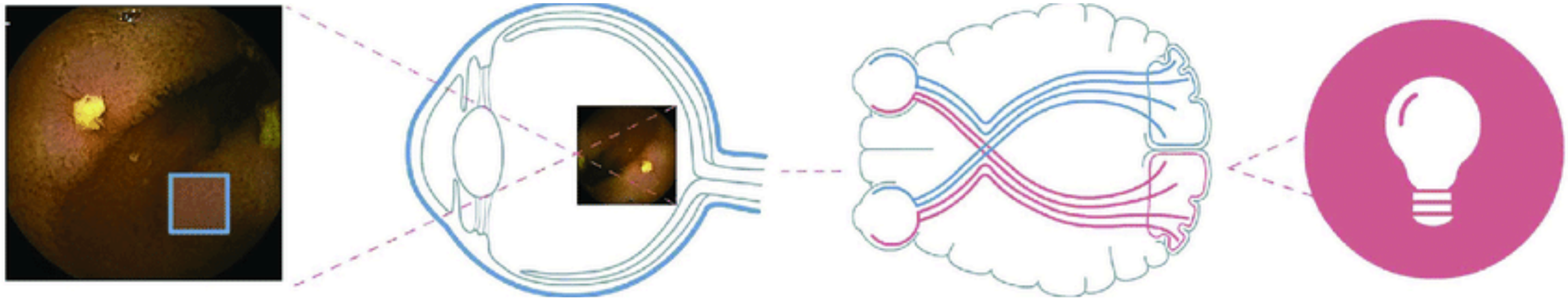
500 픽셀

개?
고양이?

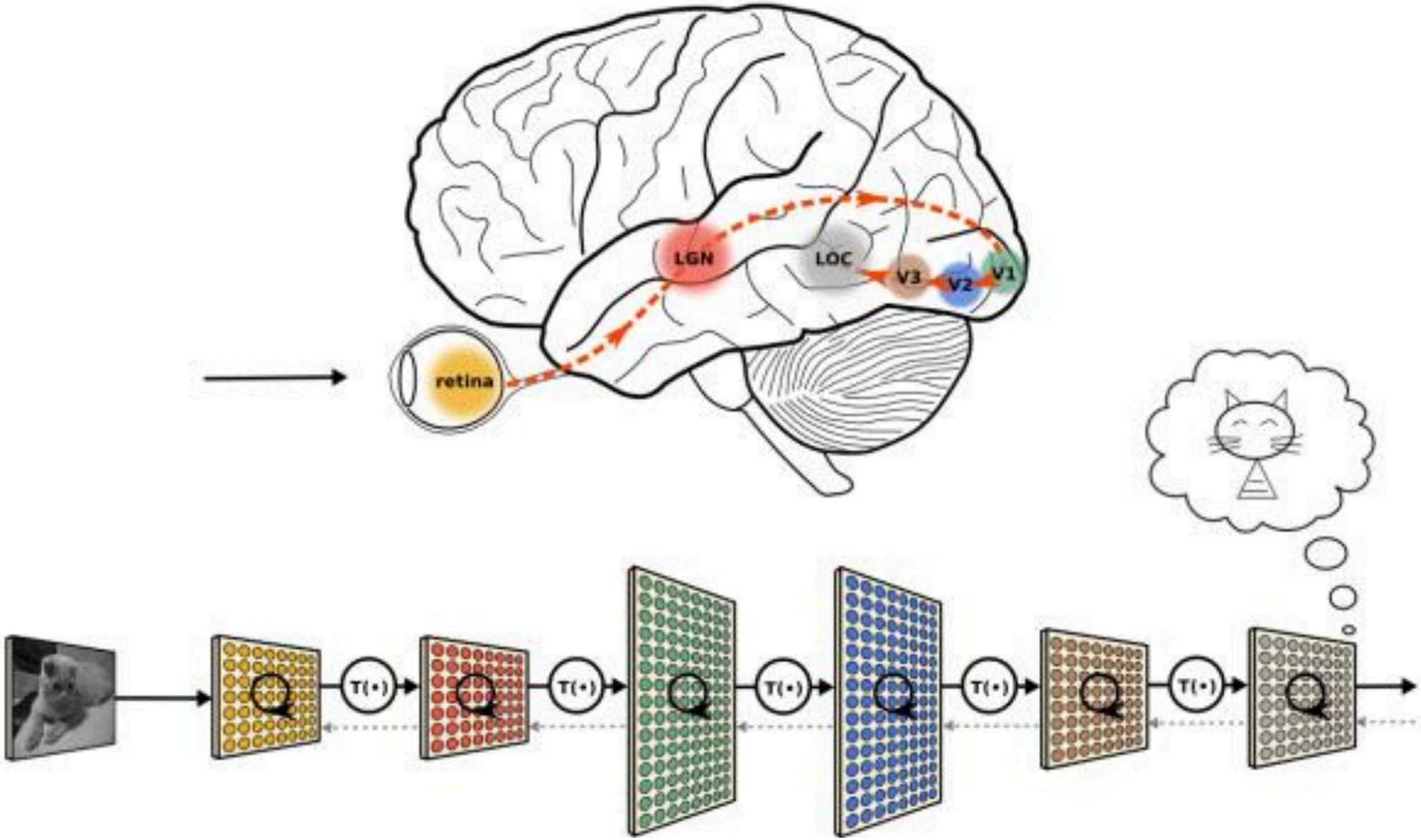
네트워크의 상태가... ?!

VISION

시각이란?

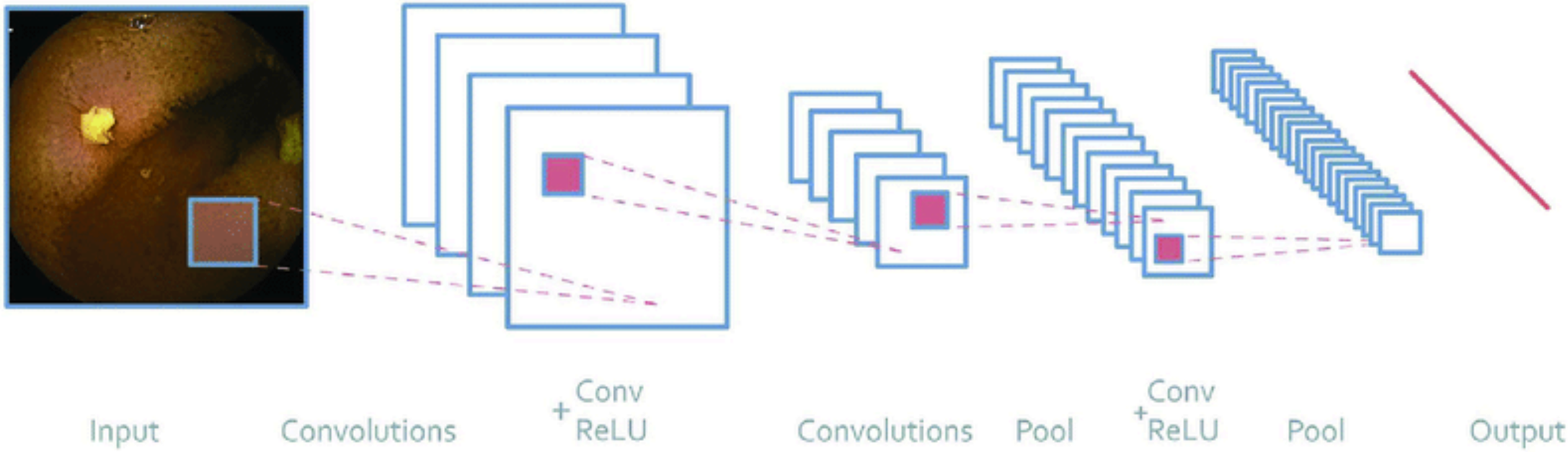


SOFFER, SHELLY, ET AL. "DEEP LEARNING FOR WIRELESS CAPSULE ENDOSCOPY:
A SYSTEMATIC REVIEW AND META-ANALYSIS." GASTROINTESTINAL ENDOSCOPY 92.4 (2020): 831-839.



https://figshare.com/articles/figure/Ventral_visual_stream/106794

CNN


Input
Convolutions
+ Conv ReLU
Convolutions
Pool
+ Conv ReLU
Pool
Output

LENET-5 (1989)

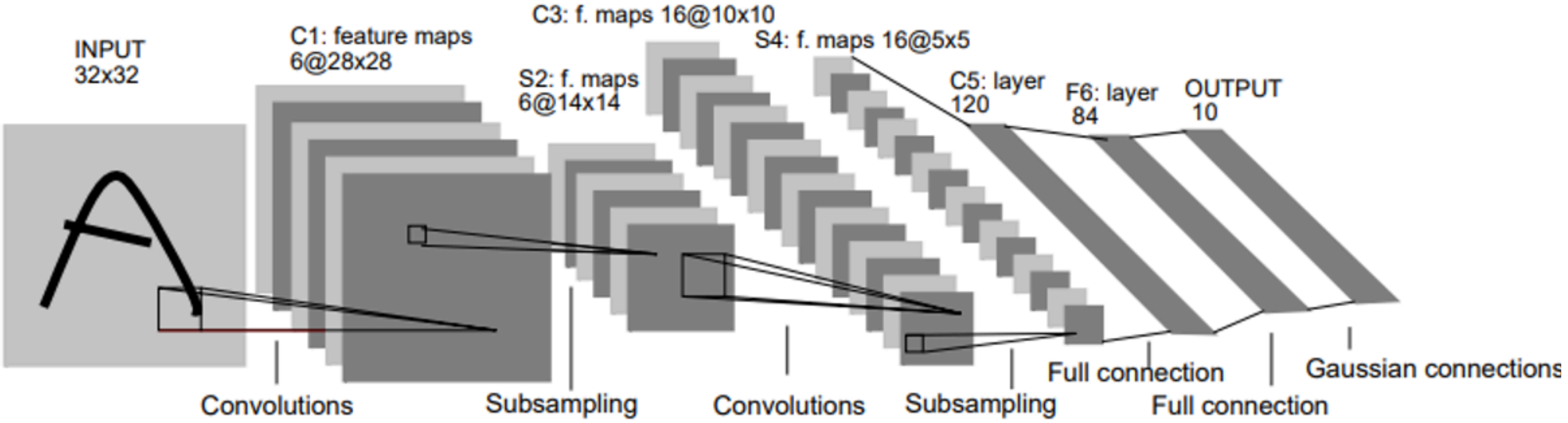
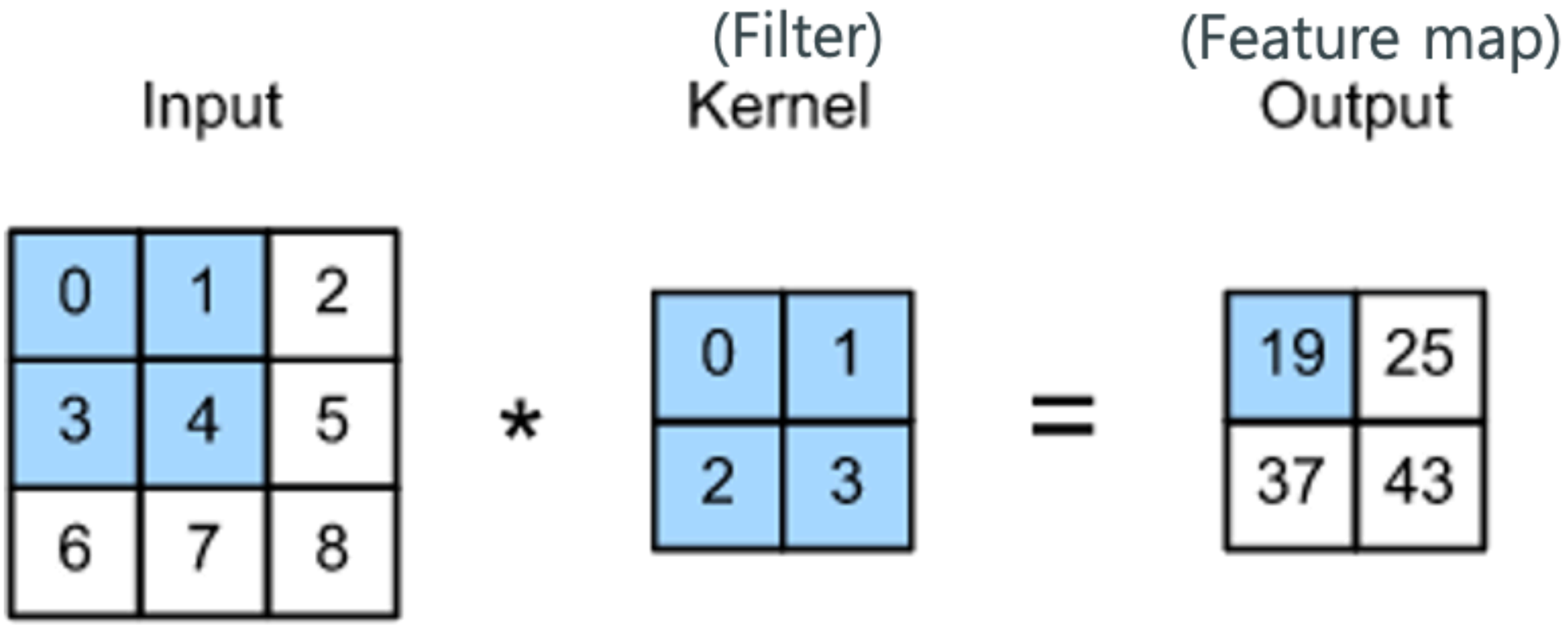


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of unit whose weights are constrained to be identical.

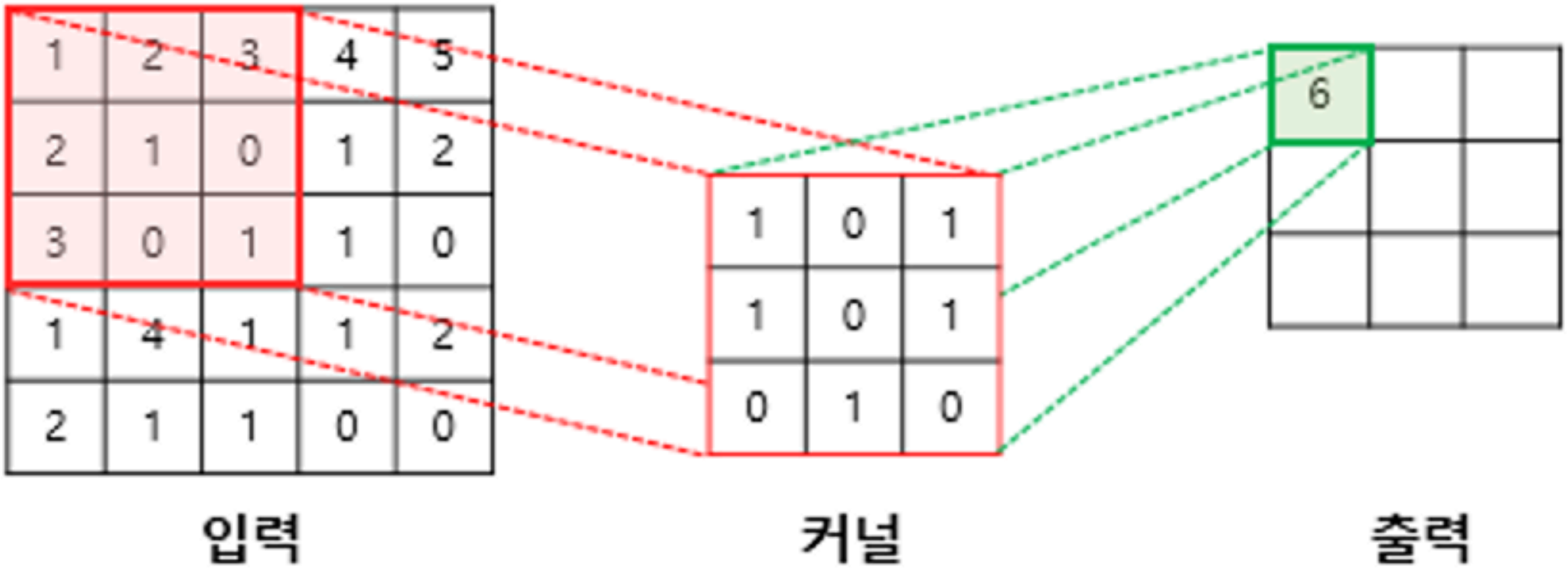
CONVOLUTION + SUBSAMPLING + FULL CONNECTION

CONVOLUTION

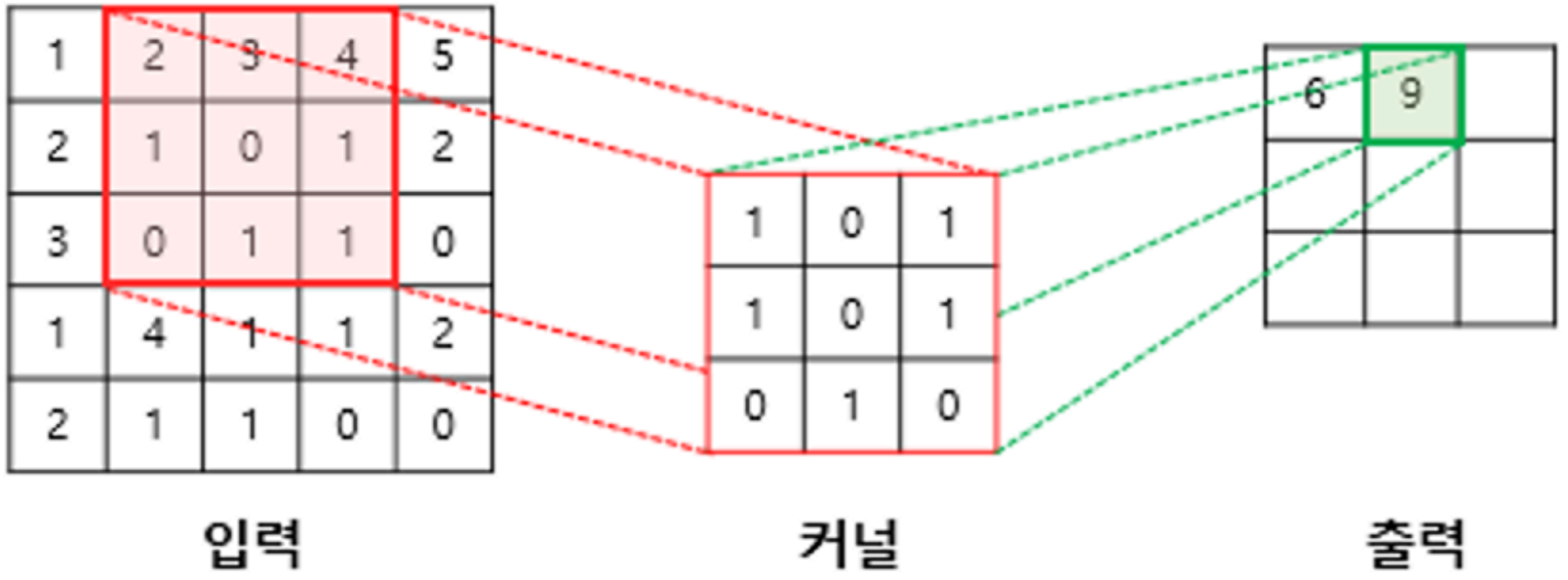


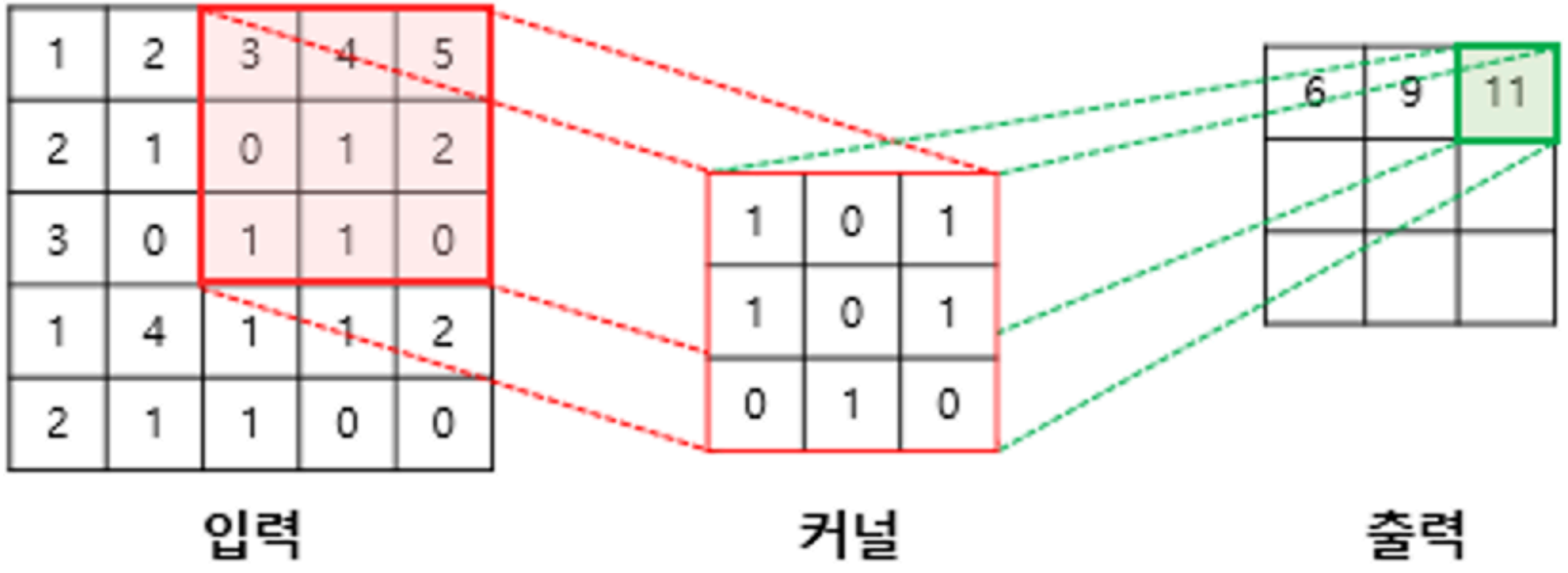
엄밀한 정의는 CORRELATION이지만... 넘어가도록 하자

CONVOLUTION LAYER

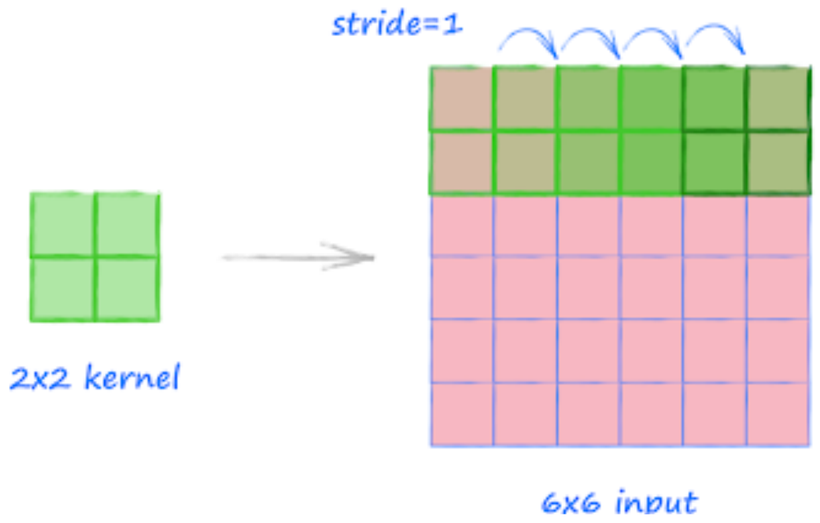


나머지 칸은 어떻게?

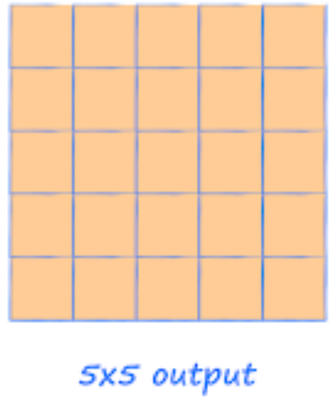

1 2 3 4 5
2 1 0 1 2
3 0 1 1 0
1 4 1 1 2
2 1 1 0 0
1 0 1
1 0 1
0 1 0
6 9
입력
커널
출력

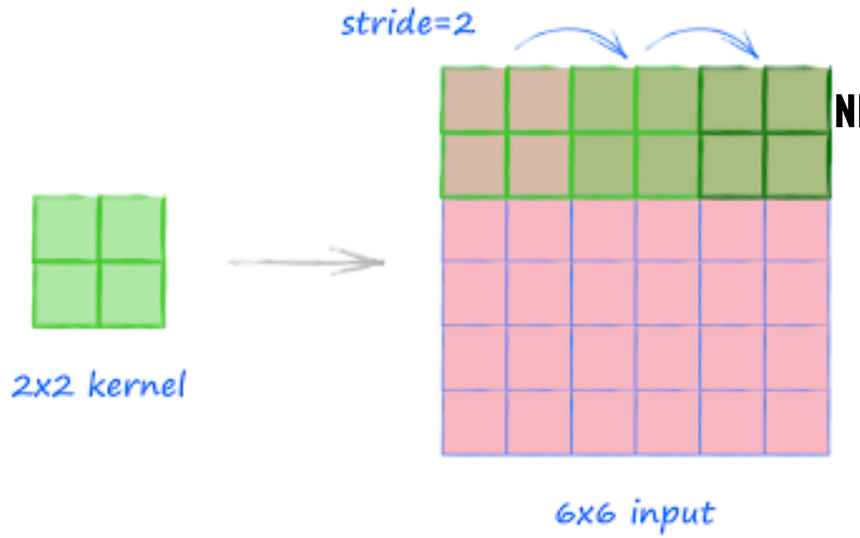

1 2 3 4 5
2 1 0 1 2
3 0 1 1 0
1 4 1 1 2
2 1 1 0 0
1 0 1
1 0 1
0 1 0
6 9 11
입력
커널
출력

STRIDE

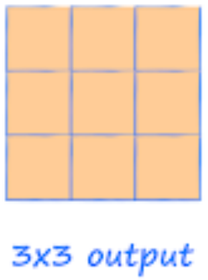


NN.CONV2D(IN_CHANNELS, OUT_CHANNELS, KERNEL_SIZE=2, STRIDE=1)



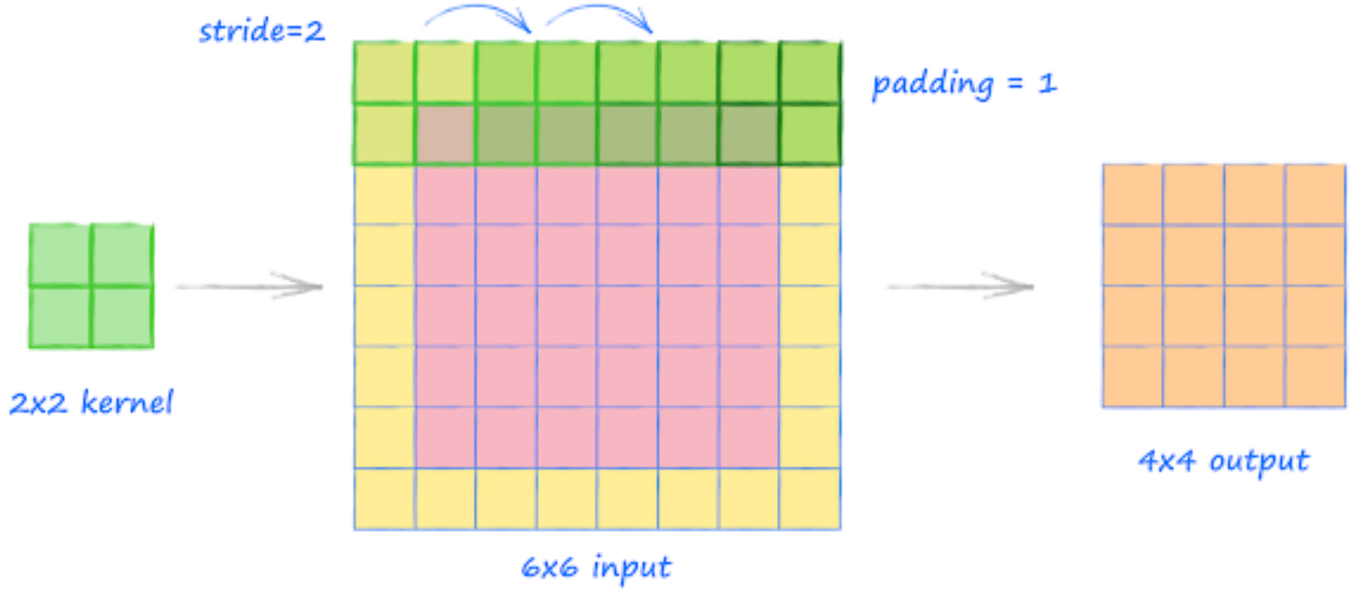


NN.CONV2D(IN_CHANNELS, OUT_CHANNELS, KERNEL_SIZE=2, STRIDE=2)



HTTPS://MAKEYOUROWNNEURALNETWORK.BLOGSPOT.COM/2020/02/CALCULATING-OUTPUT-SIZE-OF-CONVOLUTIONS.HTML

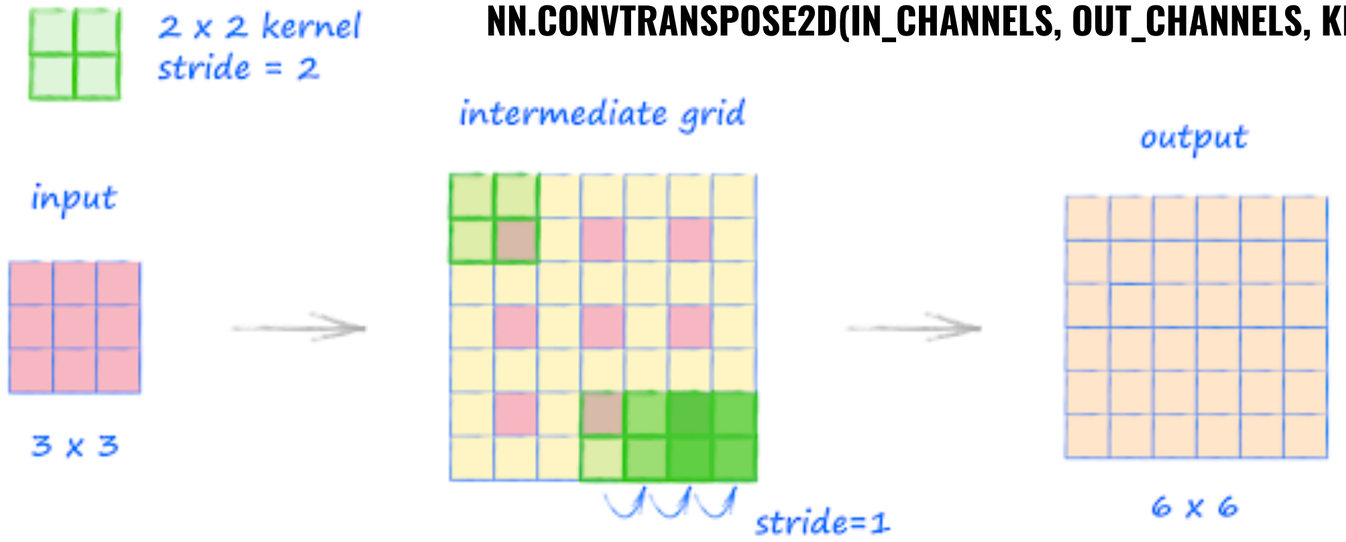
PADDING

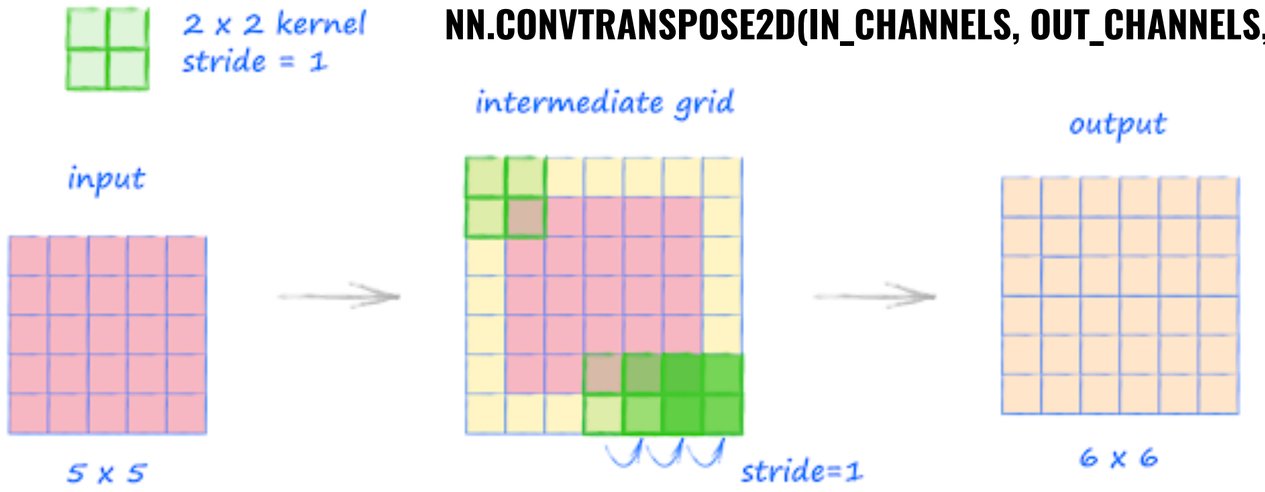


NN.CONV2D(IN_CHANNELS, OUT_CHANNELS, KERNEL_SIZE=2, STRIDE=2, PADDING=2)

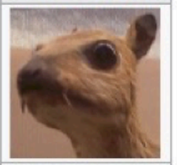
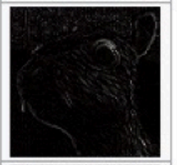
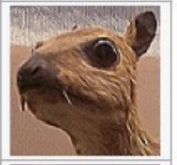
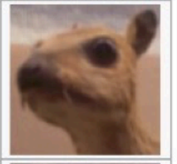
$$\text{output size} = \left\lfloor \frac{(\text{input size}) + 2*\text{padding} - (\text{kernel size} - 1) - 1}{\text{stride}} + 1 \right\rfloor$$

TRANSPOSED CONVOLUTION





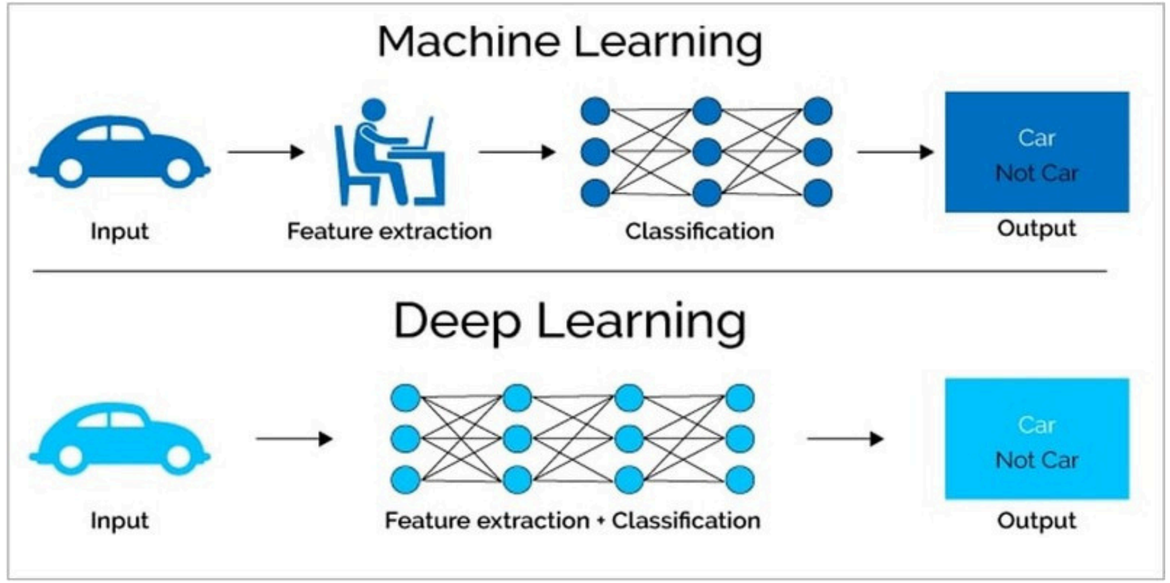
CONVOLUTION LAYER 효과

Operation	Kernel	Image result
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

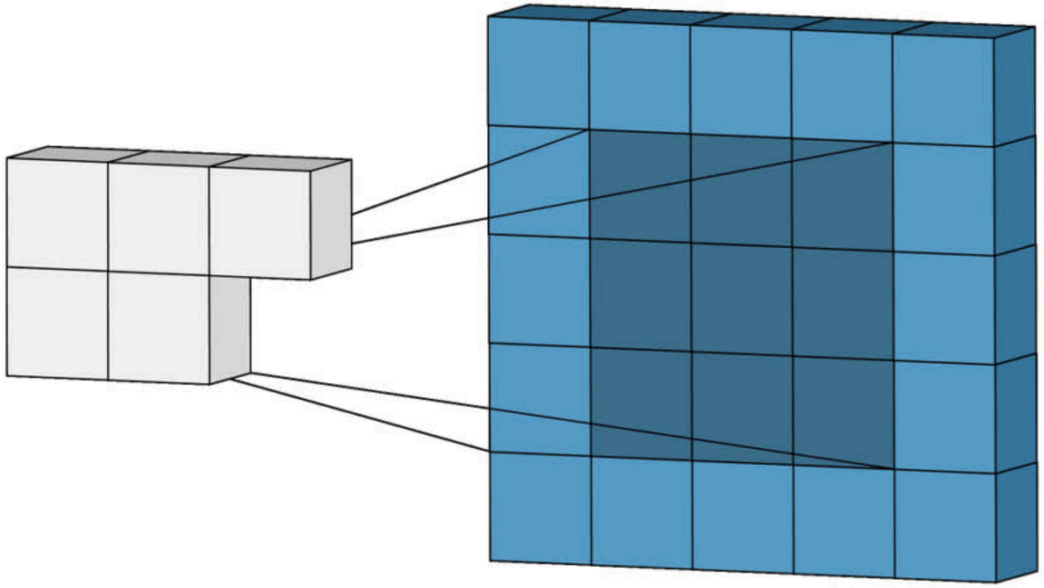
기존에 알려진 커널(필터) 효과

신경망의 일부분이라면? 학습대상

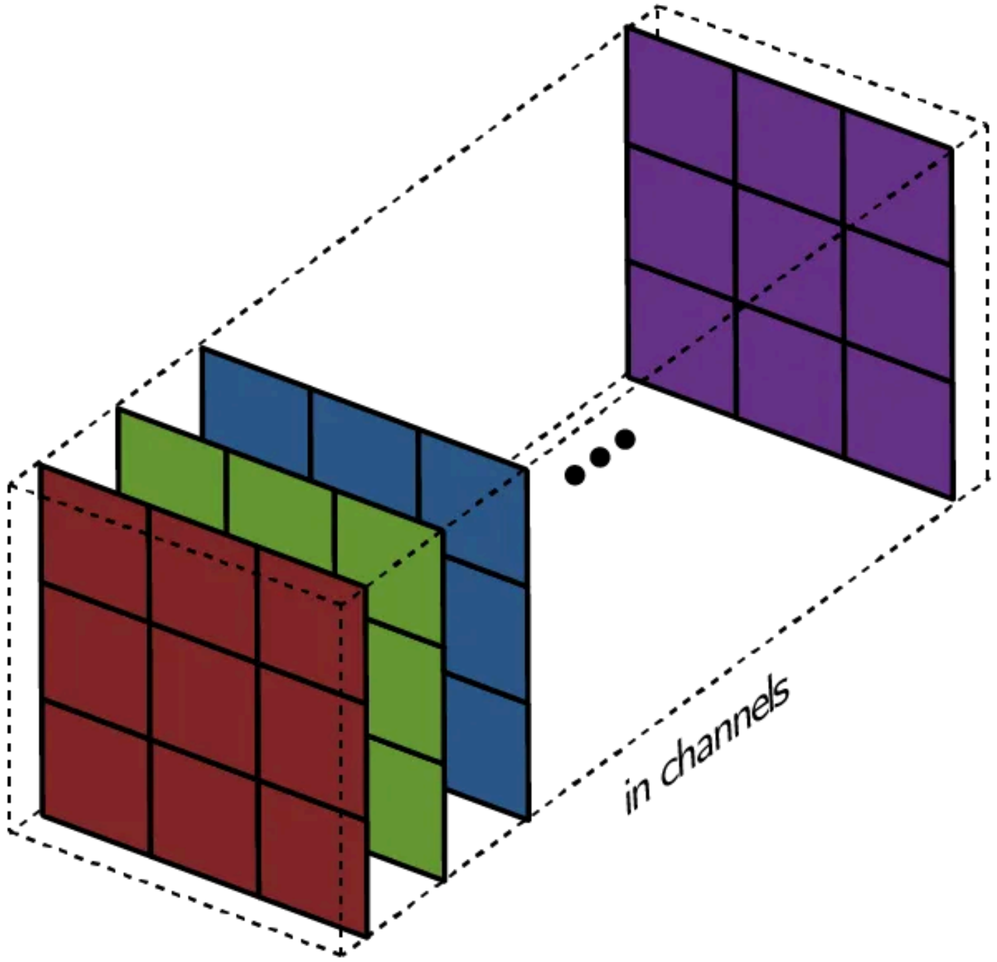
= AI가 스스로 모양에 대해 분석하게 하자



실제로 이미지는 3차원



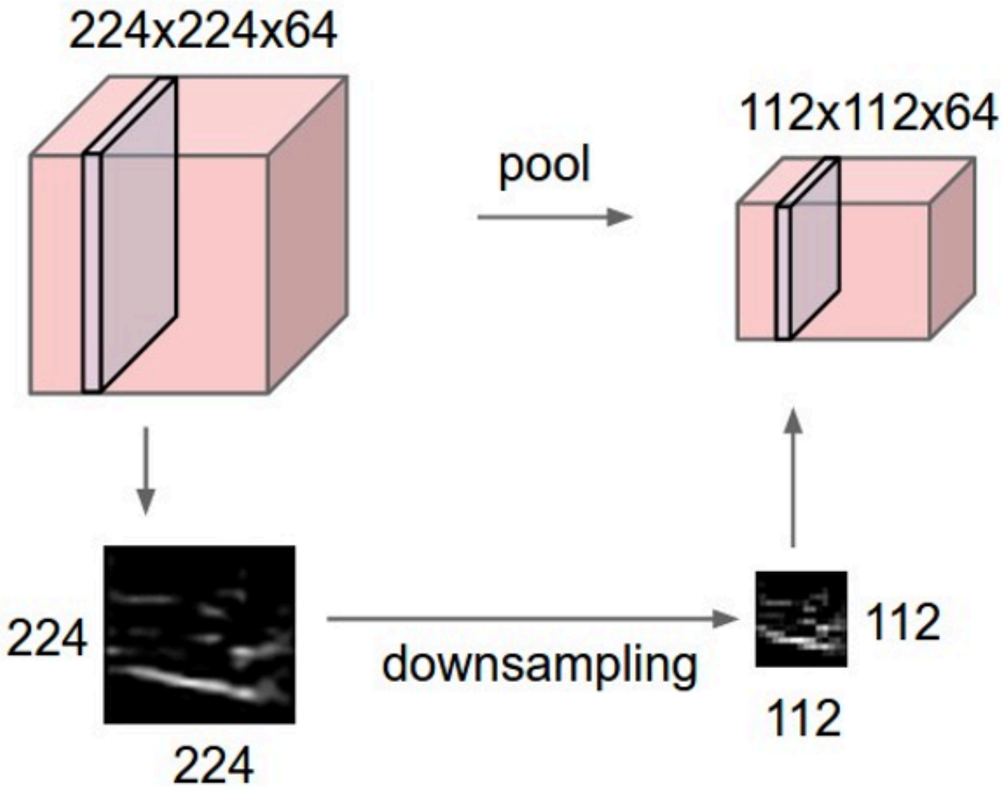
커널 하나가 하나의 결과를 생성



커널이 여러개면?!
결과가 여러장!

HTTPS://TOWARDSDATASCIENCE.COM/INTUITIVELY-UNDERSTANDING-CONVOLUTIONS-FOR-DEEP-LEARNING-1F6F42FAEE1

POOLING



Activation Map

12	20	30	0
8	12	2	0
34	70	37	7
112	100	22	12

Max Pooling

20	30
112	37

Average Pooling

13	8
79	18

공간정보를 압축!

예제

```
[ ]  import torch
     import torchvision
     import torchvision.transforms as transforms
```

The output of torchvision datasets are PILImage images of range [0, 1]. We transform them to Tensors of normalized range [-1, 1].

Note

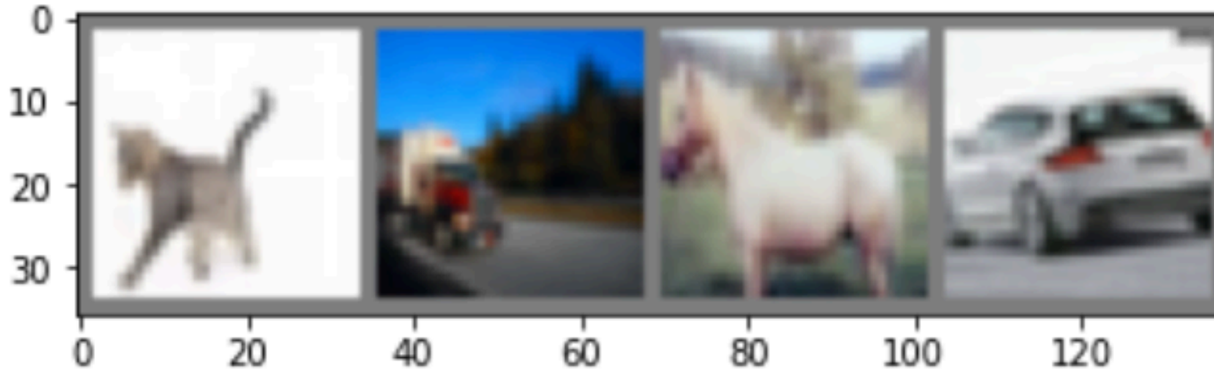
If running on Windows and you get a BrokenPipeError, try setting the num_worker of torch.utils.data.DataLoader() to 0.

```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 4

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)
```

CIFAR10

✓ 9초 오전 7:52에 완료됨

```python
import matplotlib.pyplot as plt
import numpy as np

# functions to show an image


def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


# get some random training images
dataiter = iter(trainloader)
images, labels = dataiter.next()

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```




```
cat   truck horse car
```

```python
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```

```
[ ]  import torch.optim as optim

     criterion = nn.CrossEntropyLoss()
     optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

```
     for epoch in range(2):  # loop over the dataset multiple times

         running_loss = 0.0
         for i, data in enumerate(trainloader, 0):
             # get the inputs; data is a list of [inputs, labels]
             inputs, labels = data

             # zero the parameter gradients
             optimizer.zero_grad()

             # forward + backward + optimize
             outputs = net(inputs)
             loss = criterion(outputs, labels)
             loss.backward()
             optimizer.step()

             # print statistics
             running_loss += loss.item()
             if i % 2000 == 1999:    # print every 2000 mini-batches
                 print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
                 running_loss = 0.0

     print('Finished Training')
```

```
[ ]  PATH = './cifar_net.pth'
     torch.save(net.state_dict(), PATH)
```

```
dataiter = iter(testloader)
images, labels = dataiter.next()

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in range(4)))
```

```
GroundTruth:  cat   ship  ship  plane
```

```
[ ]  net = Net()
     net.load_state_dict(torch.load(PATH))
```

```
     <All keys matched successfully>
```

Okay, now let us see what the neural network thinks these examples above are:

```
[ ]  outputs = net(images)
```

The outputs are energies for the 10 classes. The higher the energy for a class, the more the network thinks that the image is of the particular class. So, let's get the index of the highest energy:

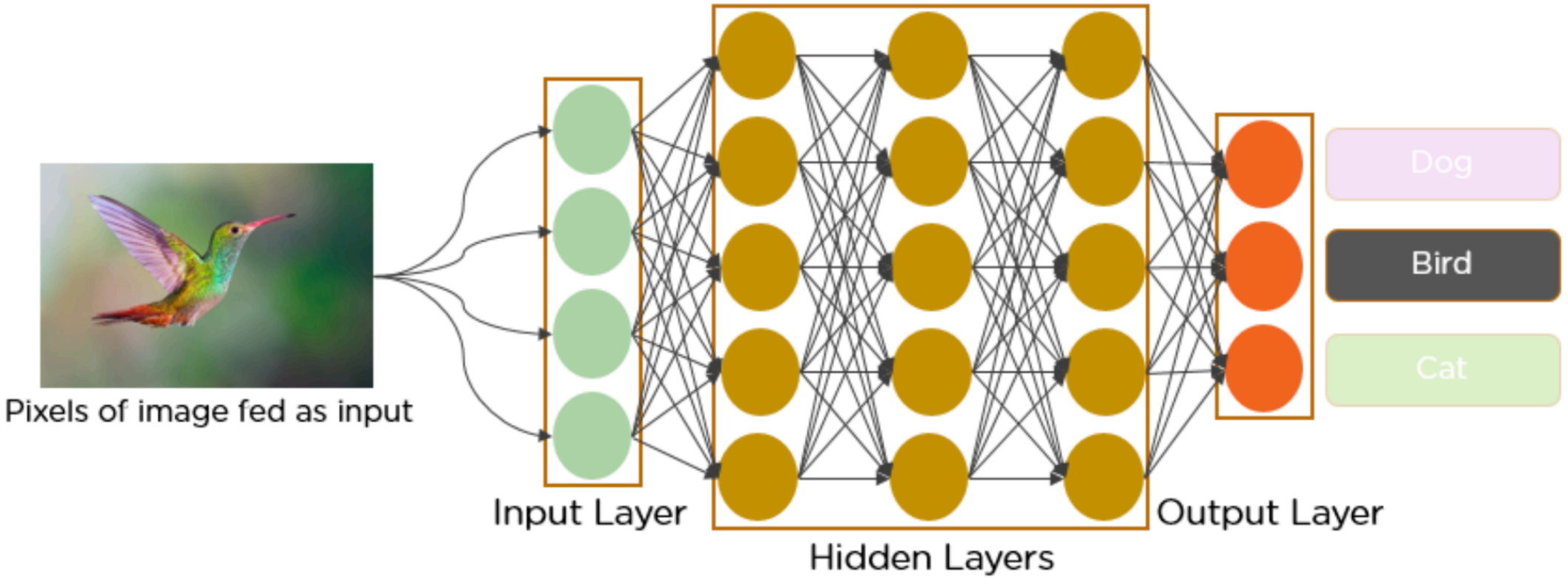
+ 코드 + 텍스트

```
[ ]  _, predicted = torch.max(outputs, 1)

     print('Predicted: ', ' '.join(f'{classes[predicted[j]]:5s}'
                                    for j in range(4)))
```

```
     Predicted:  cat   ship  ship  plane
```

왜?

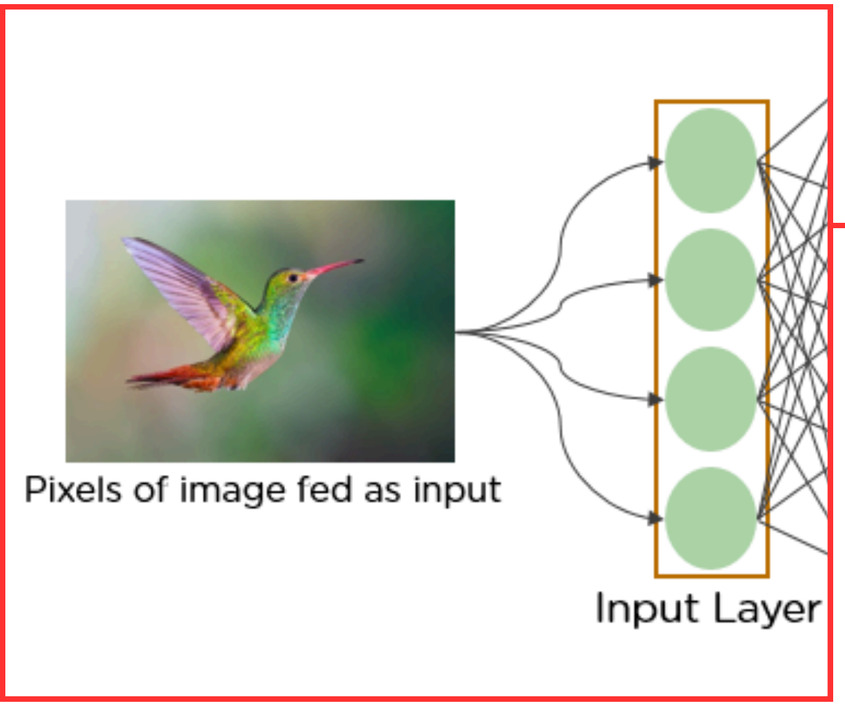
이미지를 MLP로 처리한다고 생각해보자

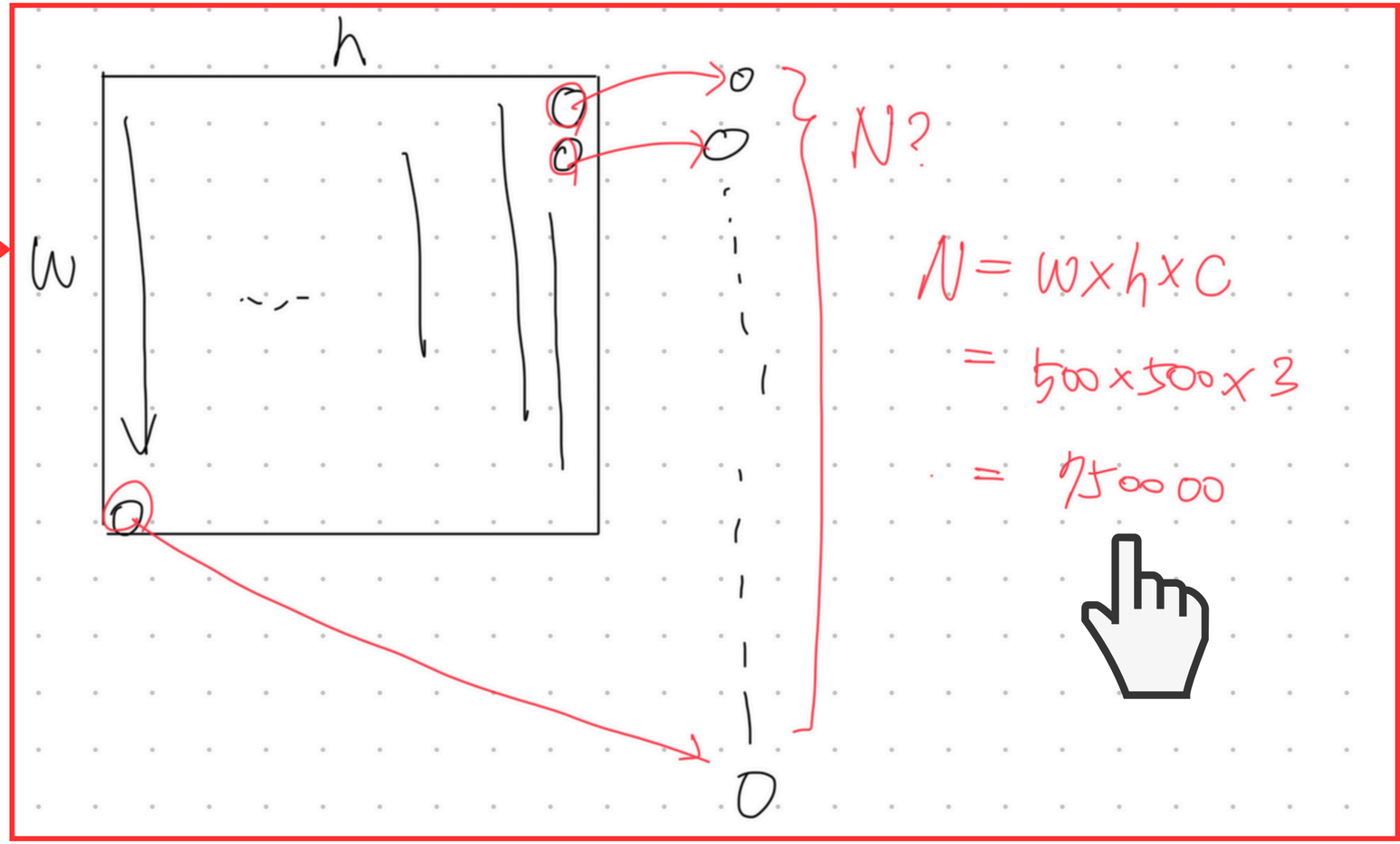


HTTPS://WWW.ANALYTICSVIDHYA.COM/BLOG/2021/05/CONVOLUTIONAL-NEURAL-NETWORKS-CNN/

왜?

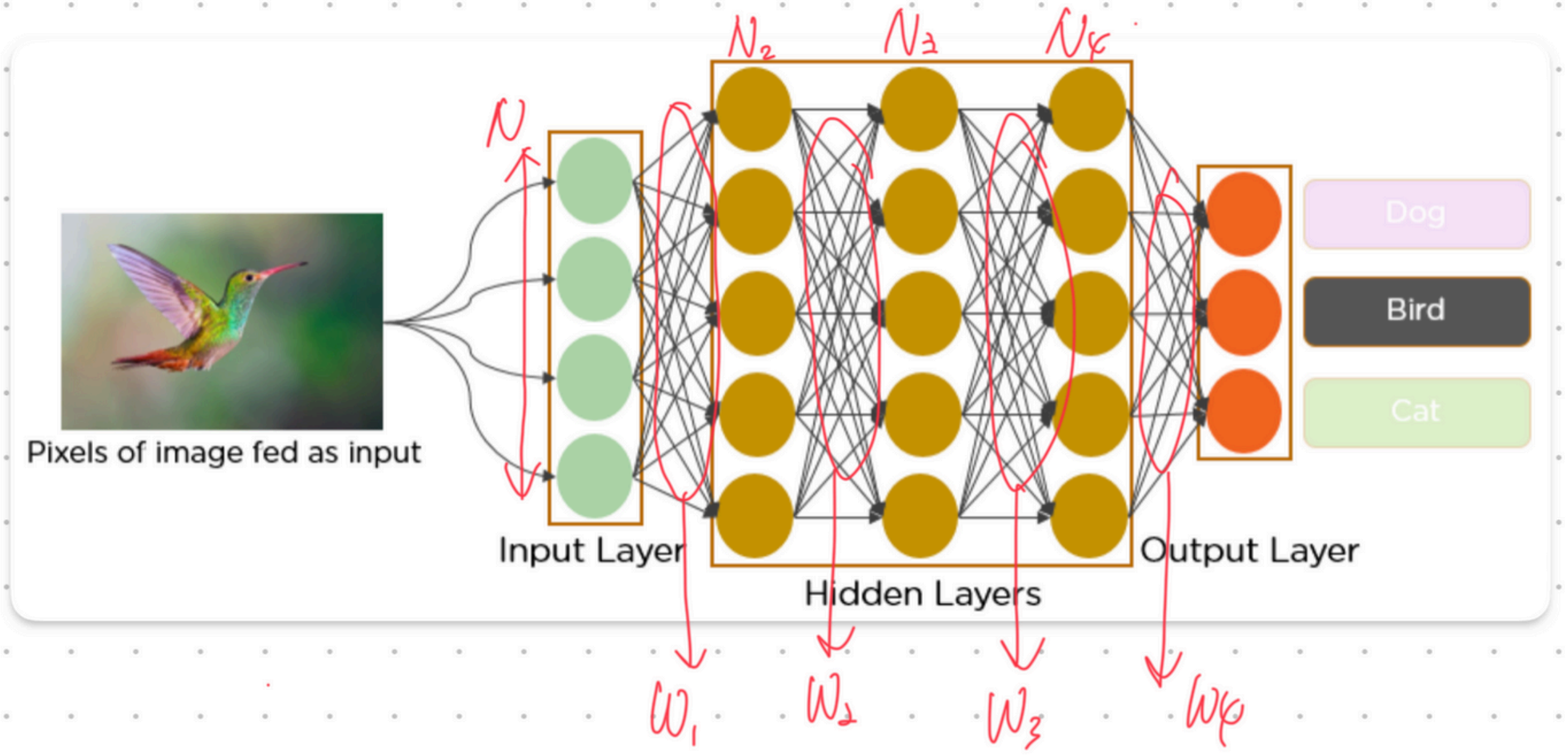
이미지를 MLP로 처리한다고 생각해보자





왜?

이미지를 MLP로 처리한다고 생각해보자



귀엽죠?

'이럴수가' 마트에서 구입한 블루베리 머핀 집에서 키우는 애완견과 판박이

【서울=뉴시스】권성근 기자 = 미국 캘리포니아 라메사에 거주하는 캘린 벨(14)은 대형 마트에서 산 블루베리 머핀을 보고 감짝 놀랐으며 그 이유는 머핀이 애완견과 너무나 닮았기 때문이라고 23일(현지시간) 허핑턴포스트가 보도했다.

뉴시스 / Dec 28, 2016

google.com

Google 치와와 머핀

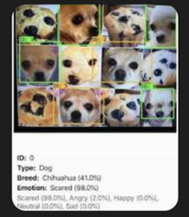
전체 이미지 쇼핑 동영상 뉴스 더보기 도구 세이프서치

인공 지능 | 인공지능강아지 | 강아지 | 초코 | 로봇 | 초코칩 | 푸들 | 쿠키 | 인간

브런치스토리
머핀과 치와와는 잊어라! 인간보다 창의적인 인공지능?!

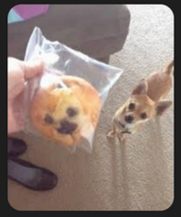
동아일보
머핀? 치와와? 아니면 래브라…

Naver Blog
AI 인공지능 기술, 머핀과 치와와를 구별하는 방…

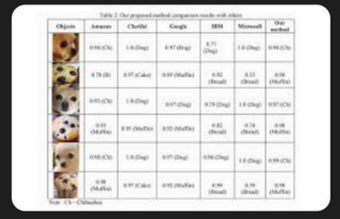

Blind
블라인드 | 블라블라: …

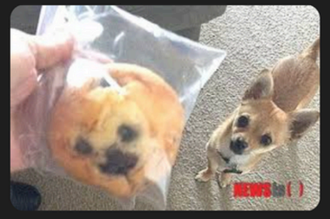
동아일보
머핀? 치와와? 아니면 래브라…

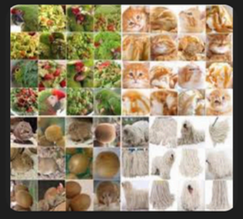
넥스트 리터러시 리뷰
친구 같은 로봇이 나쁜 말을 하는 이유 < 주니어미…


Naver Blog - 네이버
블루베리 머핀을 구웠…

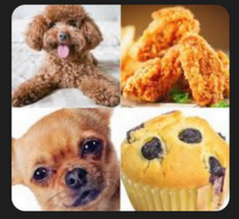
나무위키:대문
치와와 - 나무위키

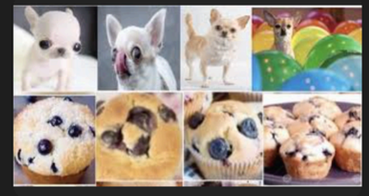

앤톡
머핀과 치와와를 구별하기 시작하는 AI - 유머…


뉴시스
이럴수가' 마트에서 구입한 블루베리 머핀 집…

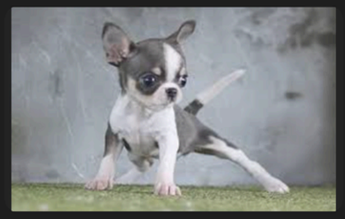

동아일보
머핀? 치와와? 아니면 래브라두…

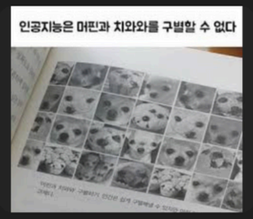
Daum 카페
드디어 머핀과 치와와를 …


ZUM 뉴스
AI 사피엔스 시대]사람 눈처럼 …


넥스트 리터러시 리뷰
친구 같은 로봇이 나쁜 말을 하는 이유 < 주니어미…

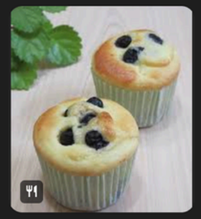
일요신문
개야 빵이야? | 일요신문

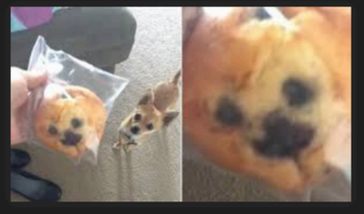

애니멀즈
치와와분양 - 치와와 - 머핀 - 애니멀즈

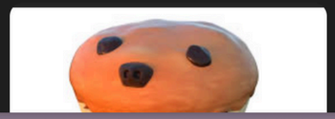


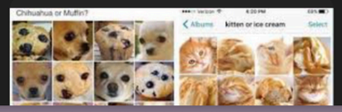
Daum 카페
인공지능은 머핀과 치와와를 구별할 …

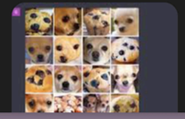
NetApp
Seeing the future with AI

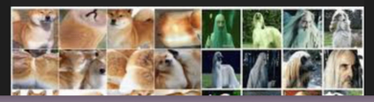

만개의레시피
블루베리머핀


애니멀플래닛
주인님, 그 녀석은 가짜예요ㅠㅠ"…블루베리 머핀한…



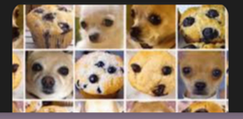


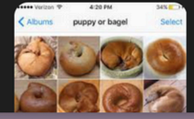


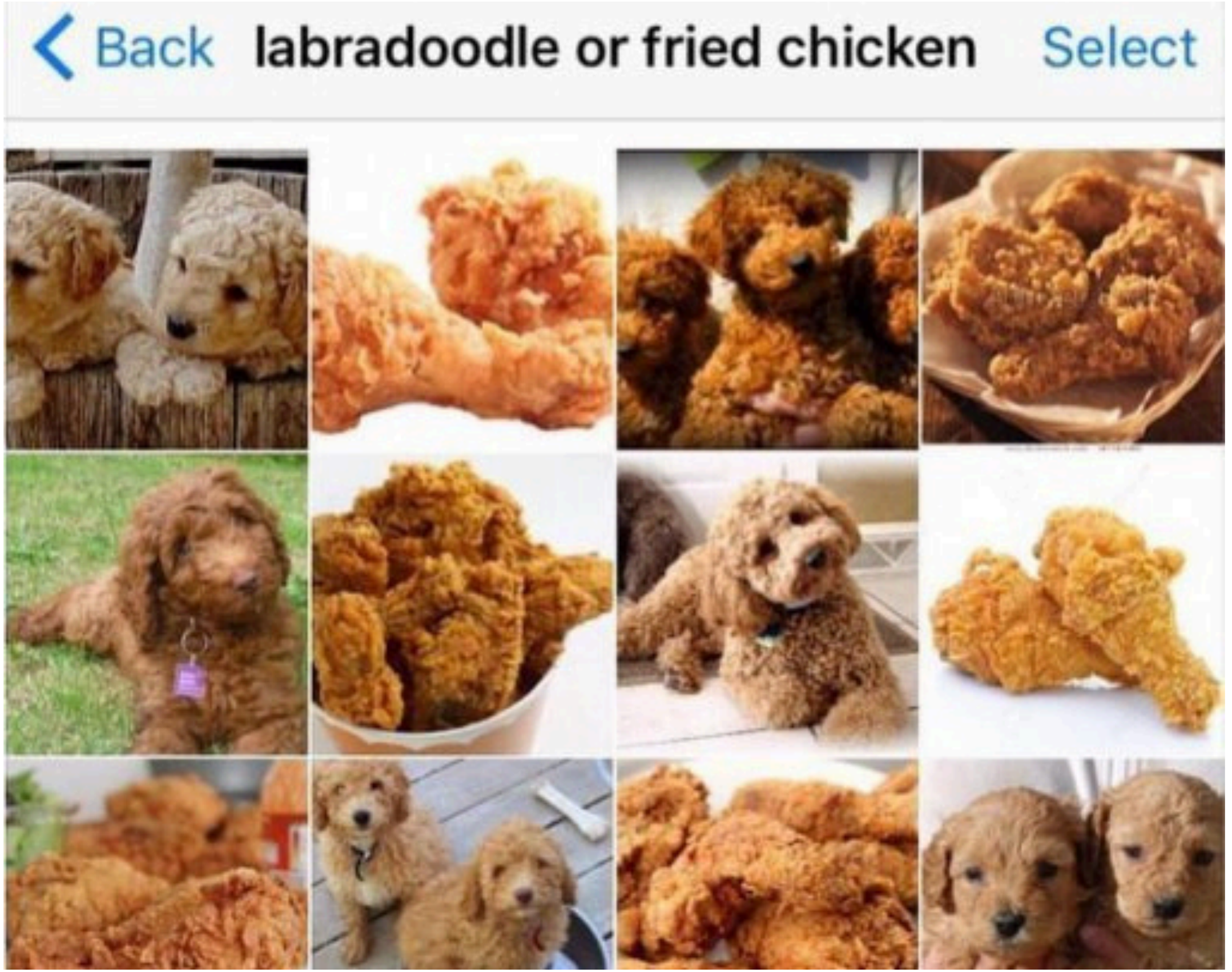


관련 검색어

인공지능 치와와 머핀







HTTPS://WWW.DONGA.COM/NEWS/IT/ARTICLE/ALL/20160317/77051739/3

치와와 vs 머핀. 인공지능은 과연?

Qualcomm사에서 2018년 AI 전문기업
Clarifai에 의뢰한 결과..
: 정확도 : 95.8%로 구분에 성공함.

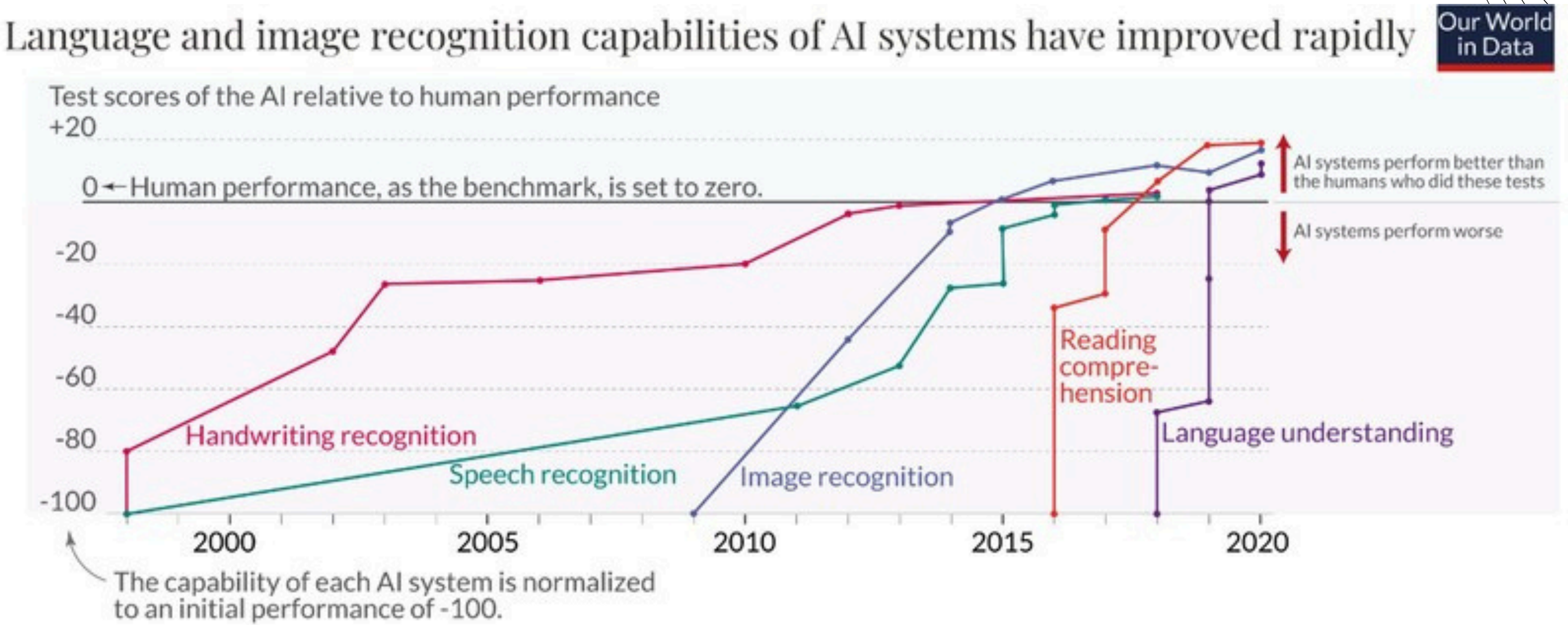
모라벡의 역설

1970년대, 미국의 로봇 공학자인 한스 모라벡(HANS MORAVEC)

‘어려운 일은 쉽고, 쉬운 일은 어렵다.
(HARD PROBLEMS ARE EASY AND EASY PROBLEMS ARE HARD.)’

라는 표현으로 컴퓨터와 인간의 능력 차이를 역설적으로 표현하였다.

인간의 능력을 넘다



Data source: Kiela et al. (2021) – Dynabench: Rethinking Benchmarking in NLP
OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the author Max Roser

이미지란?

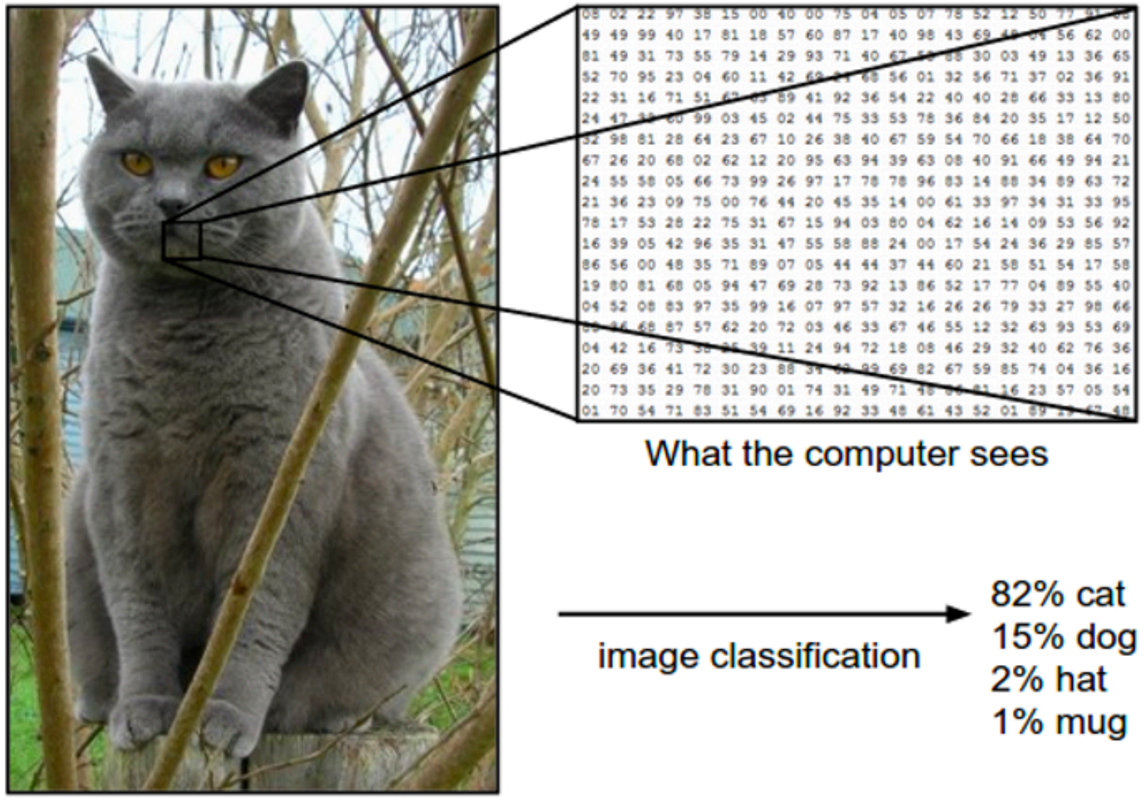
사전

정의 출처: Oxford Languages · 자세히 알아보기

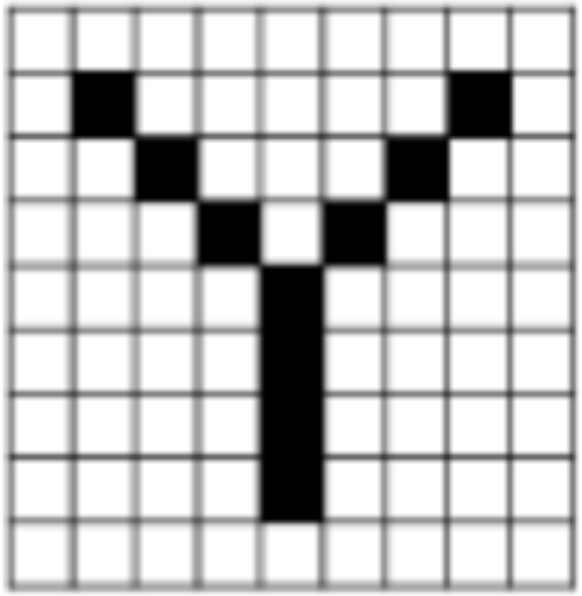
이미지, image

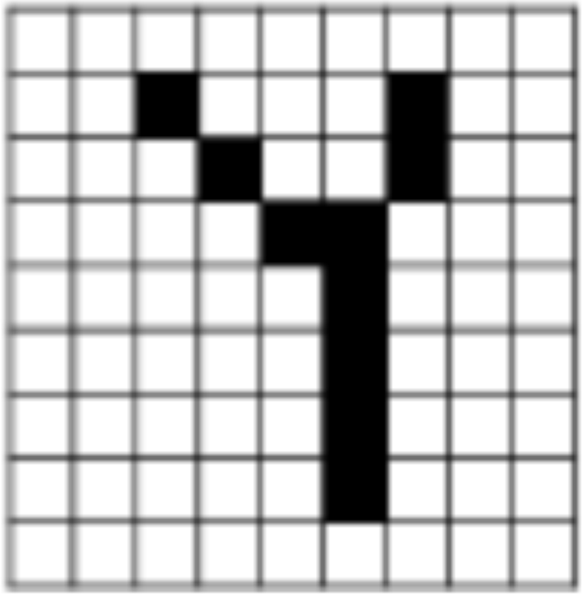
명사

1. = 심상(心象).
   "~가 떠오르다"
2. 어떤 사물이나 사람에게서 받는 인상. 순화어는 `인상'.
   "외국인에게 한국에 대한 좋은 ~를 심어 주자"



문제는 이미지라는 데이터의 다양성!!

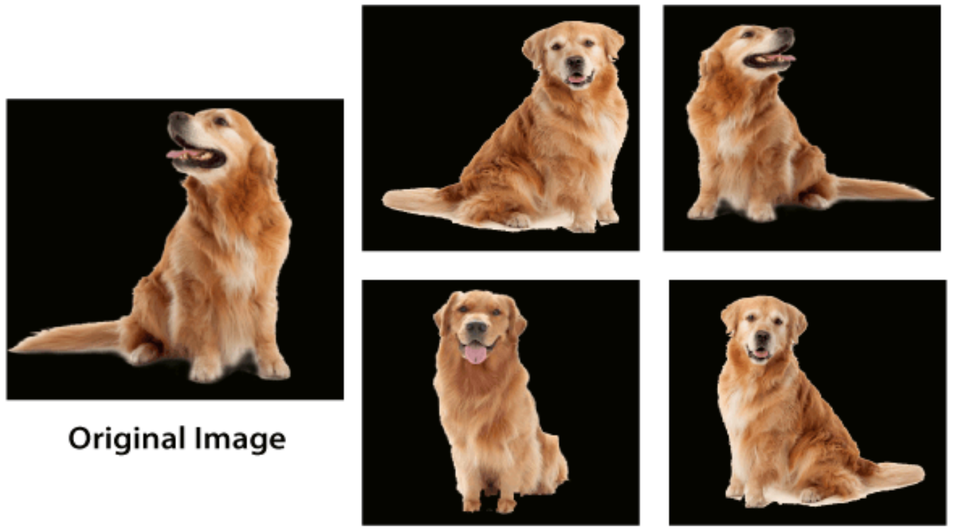




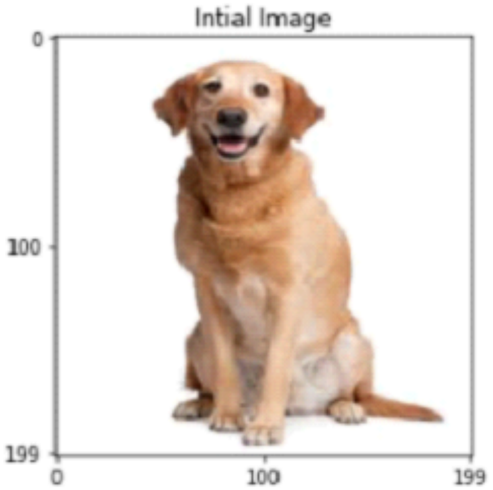
높은 정보량! 그러나 다루기 힘들다!

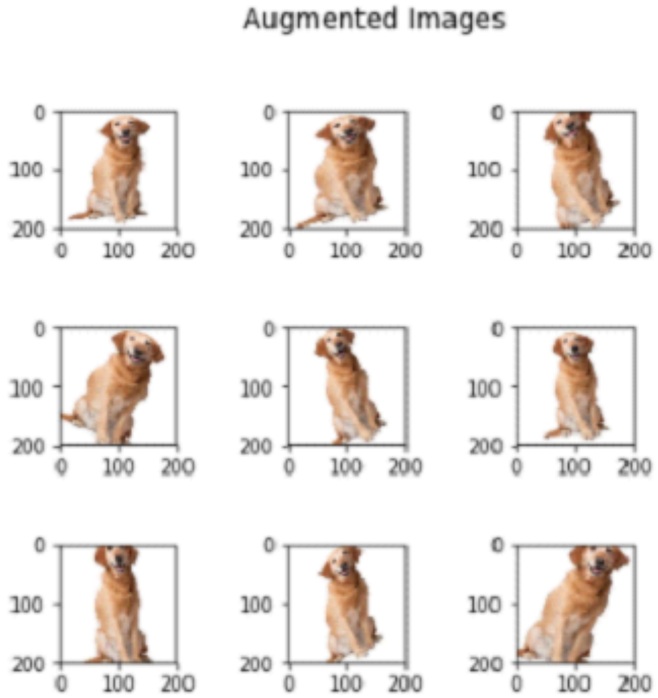
Data augmentation (증강)

이미지는 정보가 너무 풍부해서...



다양한 목표대상을
다양한 자세로...
너무... 많은데?

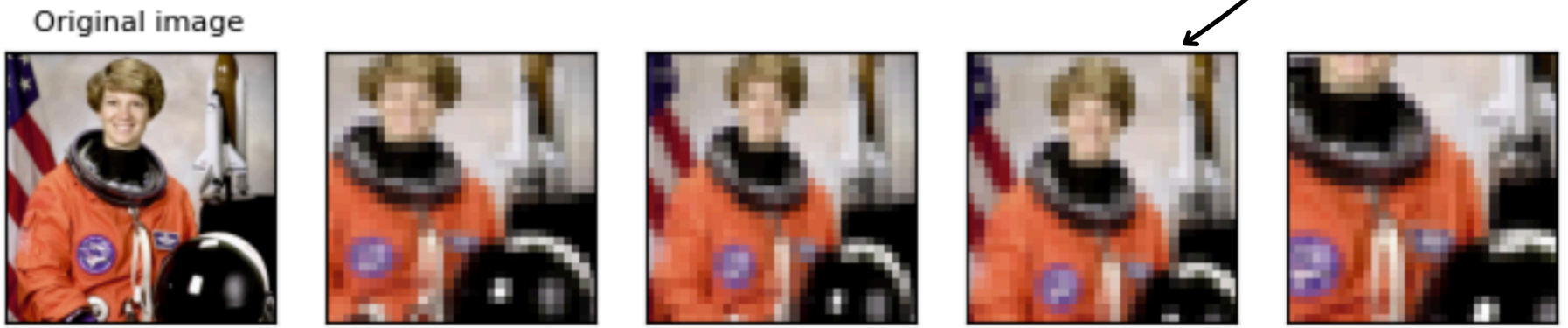




그러면.. 한개를 여러개처럼 쓰면?

https://pytorch.org/vision/0.13/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py

```python
# Image Classification
import torch
from torchvision.transforms import v2

H, W = 32, 32
img = torch.randint(0, 256, size=(3, H, W), dtype=torch.uint8)

transforms = v2.Compose([
    v2.RandomResizedCrop(size=(224, 224), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
img = transforms(img)
```



Q. 실제로 효과가 있나 실험해보는건 어때요?

사용방법

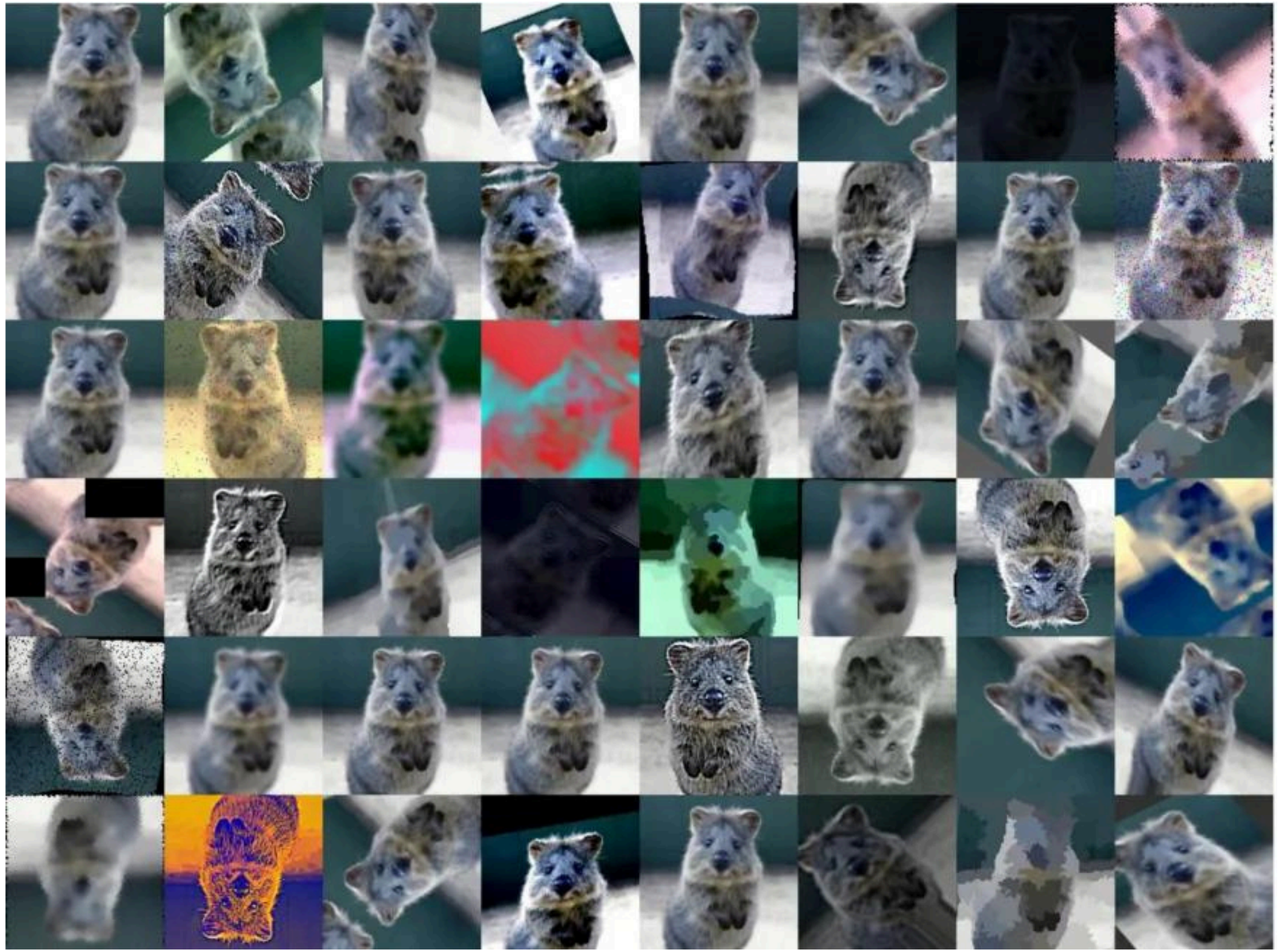
```
image_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor()
])
```

```
# 이미지 폴더로부터 데이터를 로드합니다.
transform_dataset = ImageFolder(root='tmp/PetImages',
                                transform=image_transform)
```

```
# 데이터 로더를 생성합니다.
transform_loader = DataLoader(transform_dataset,
```

HTTPS://TEDDYLEE777.GITHUB.IO/PYTORCH/PYTORCH-IMAGE-TRANSFORMS/

https://pytorch.org/vision/main/transforms.html



aleju/**imgaug**

Image augmentation for machine learning experiments.

25 Contributors · 26k Used by · 15k Stars · 2k Forks

aleju/imgaug: Image augmentation for machine learning experiments.

Image augmentation for machine learning experiments. - aleju/imgaug

GitHub

Albumentations

Albumentations: Fast Image Augmentation Library

70+ transforms • Optimized for CV & DL Workflows • Easy Integration

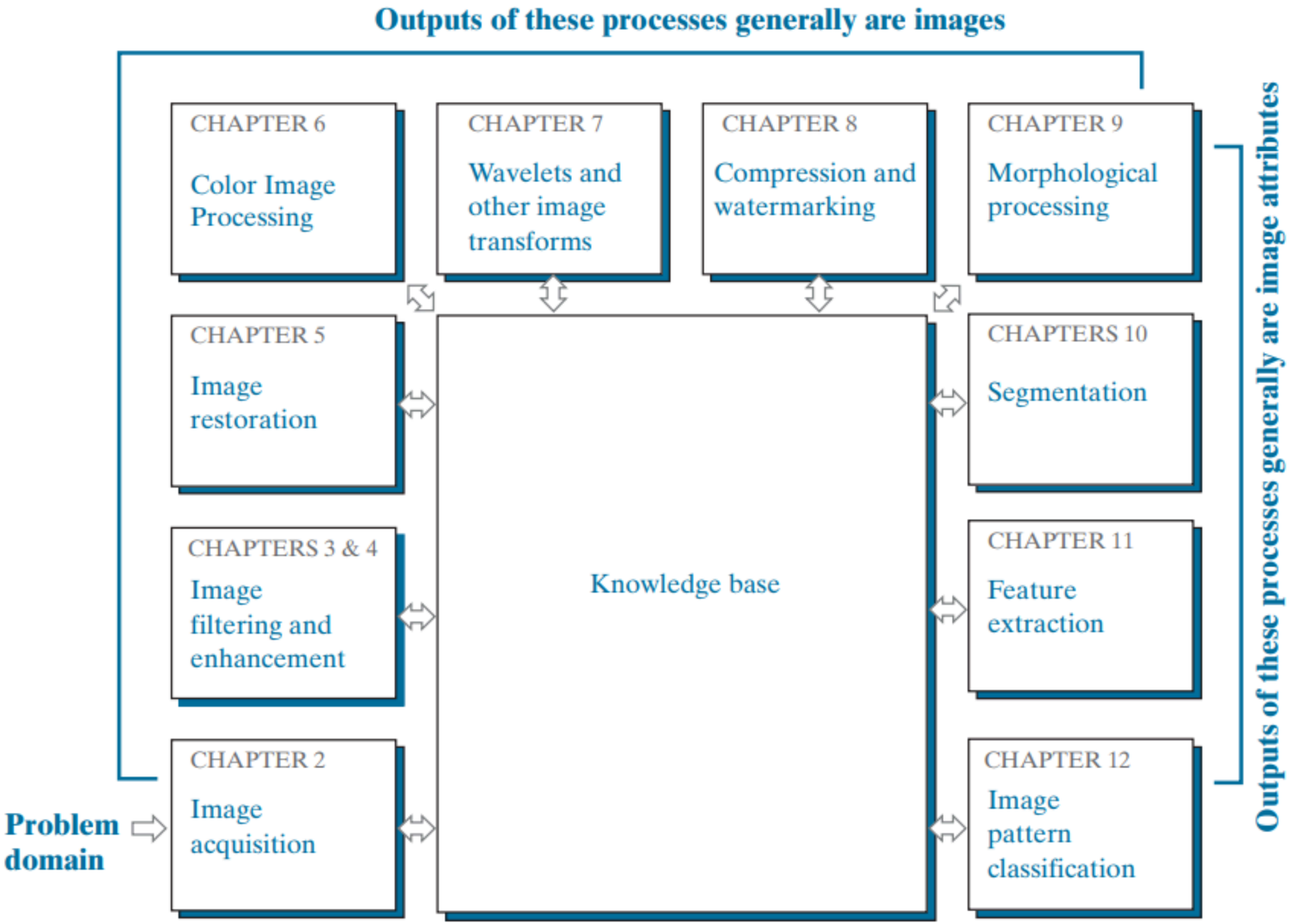
fast and flexible image augmentations

Improve computer vision models with Albumentations, the fast and flexible Python library for high-performance image augmentation. Supports images, masks, bounding boxes, keypoints & easy framework...

Albumentations

HTTPS://TOWARDSDATASCIENCE.COM/GETTING-STARTED-WITH-ALBUMENTATION-WINNING-DEEP-LEARNING-IMAGE-AUGMENTATION-TECHNIQUE-IN-PYTORCH-47AABA0EE3F8

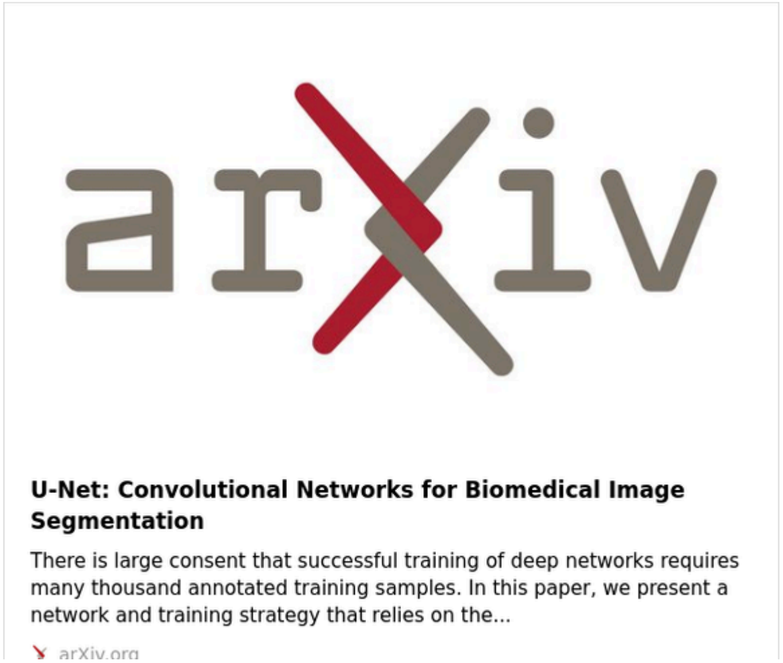
사실 이미지 프로세싱은 엄청나게 복잡합니다.

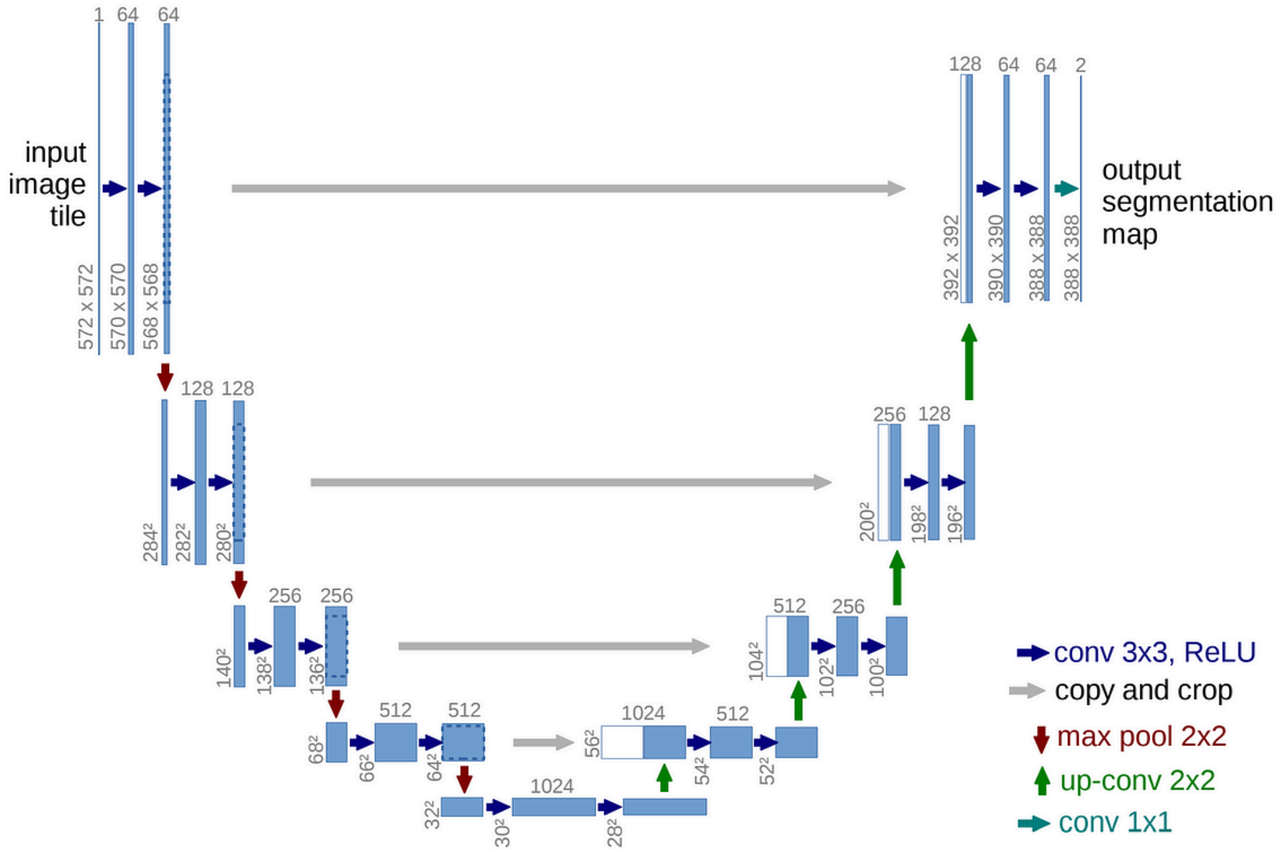


HTTPS://MEDIUM.COM/@MUHAMMAD.A0625/FUNDAMENTAL-STEPS-IN-DIGITAL-IMAGE-PROCESSING-EA2BA2A5B079

U-net

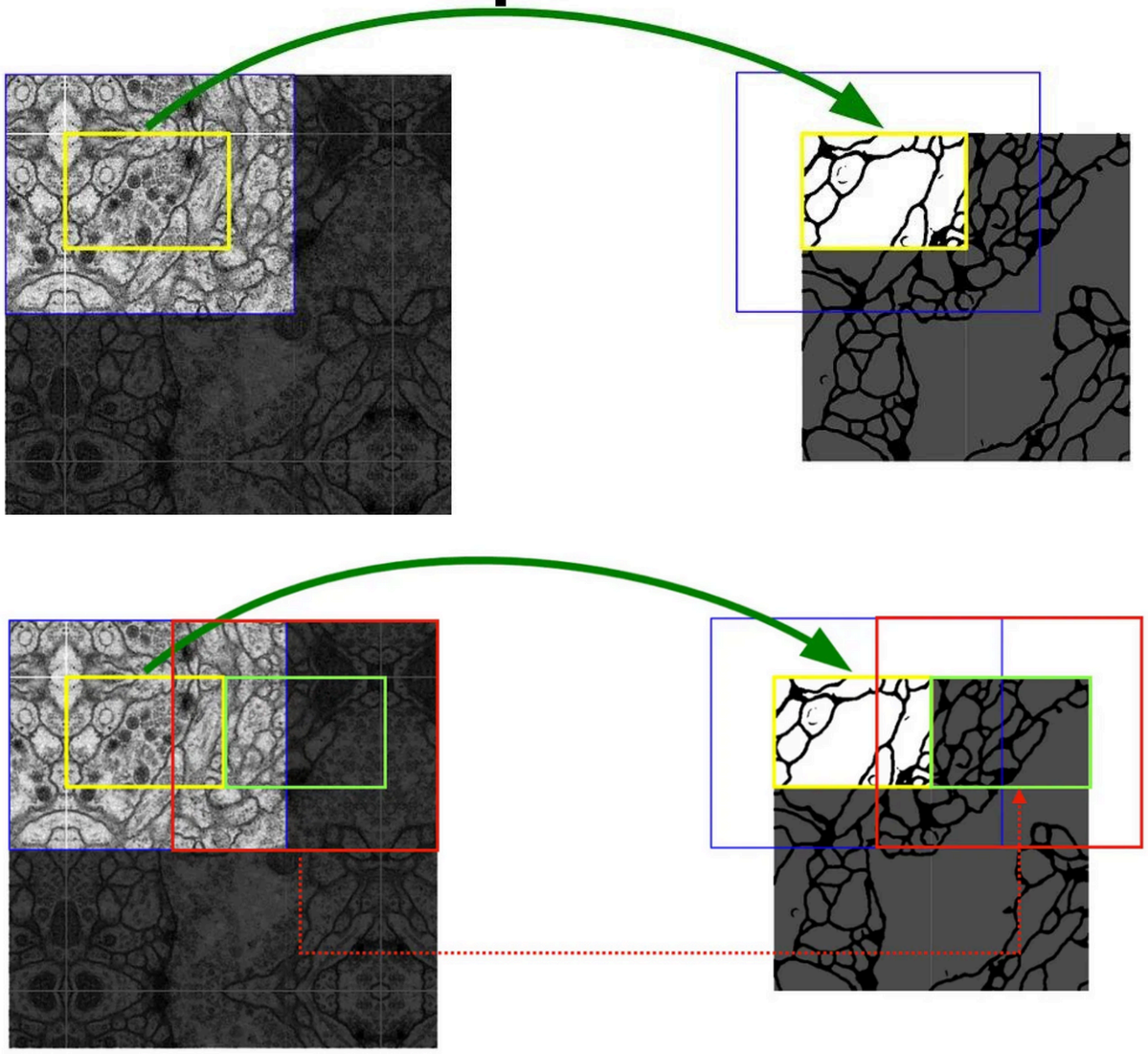
u자 모양이라하여 U-net
의료 데이터에서 많이 사용



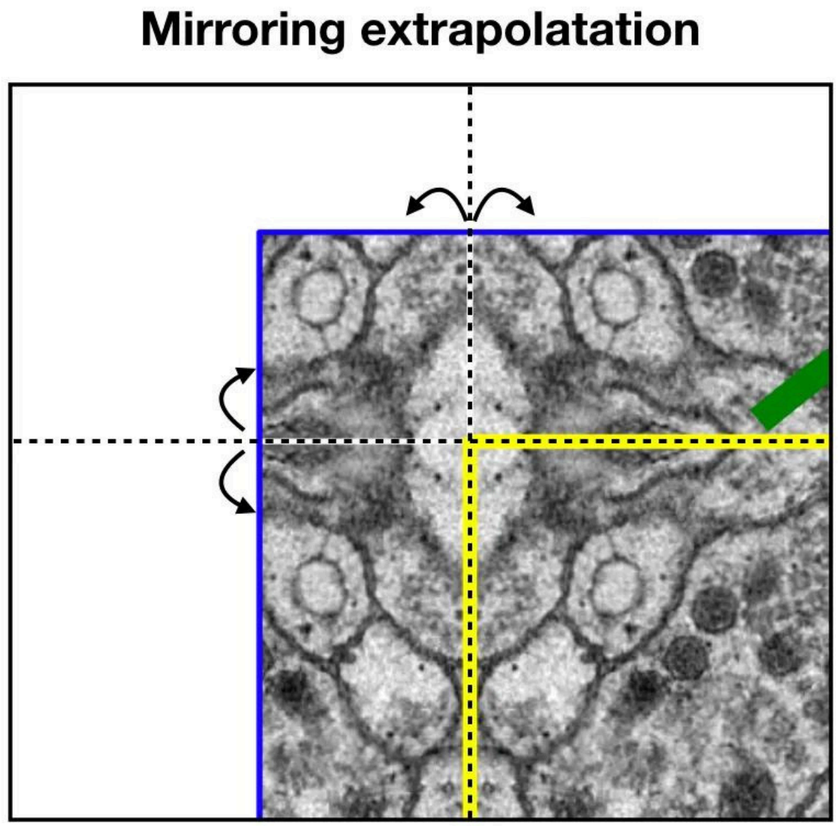


https://kuklife.tistory.com/119

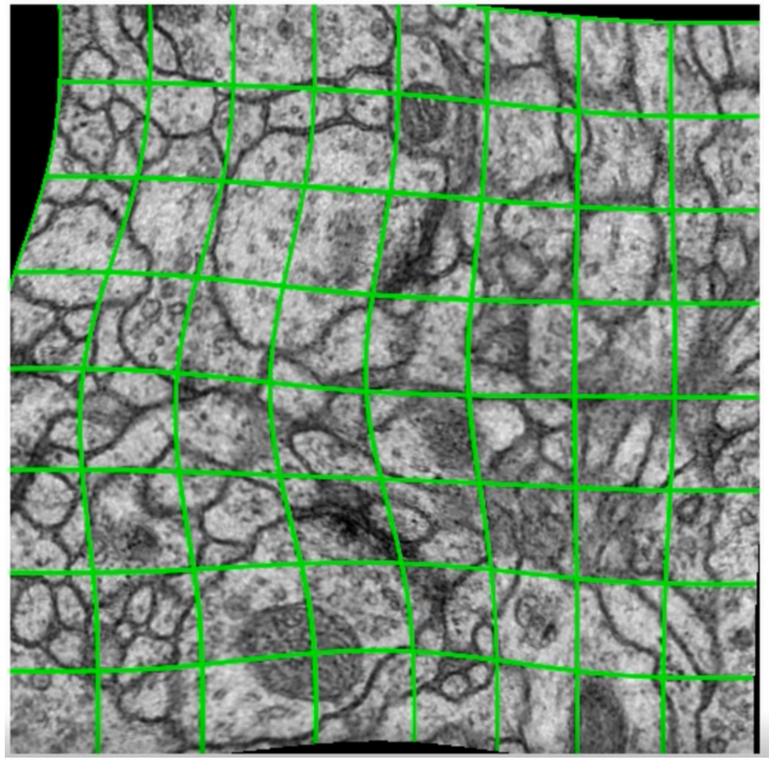
Overlap

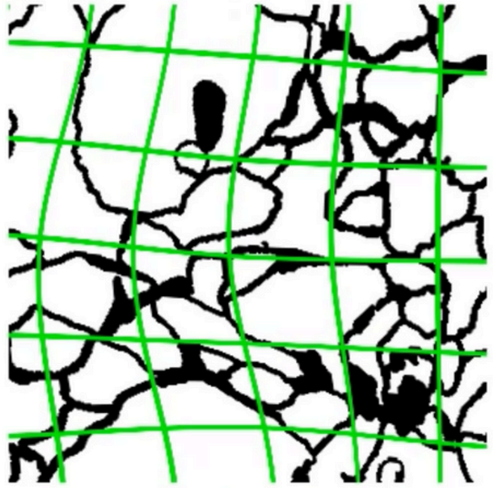


Mirror



Augmentation

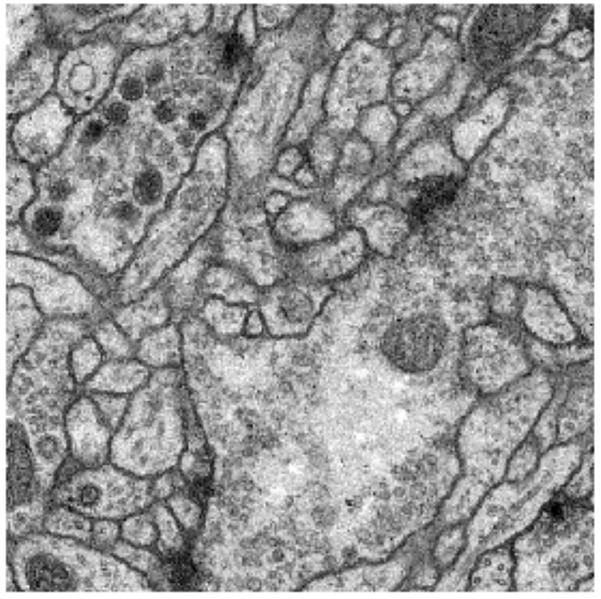


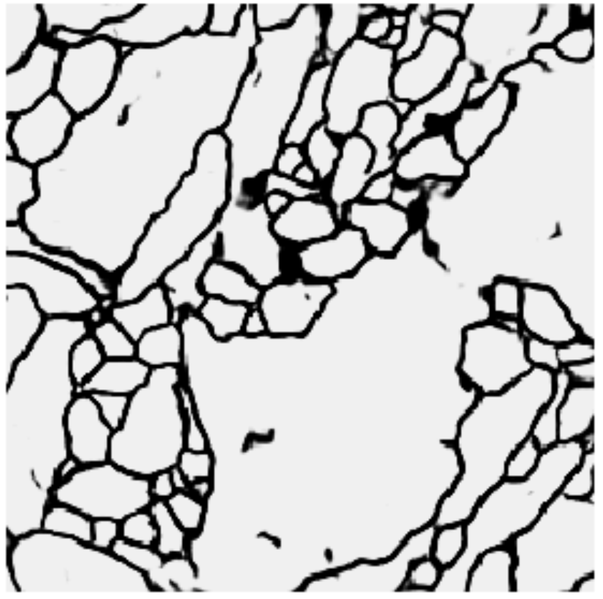


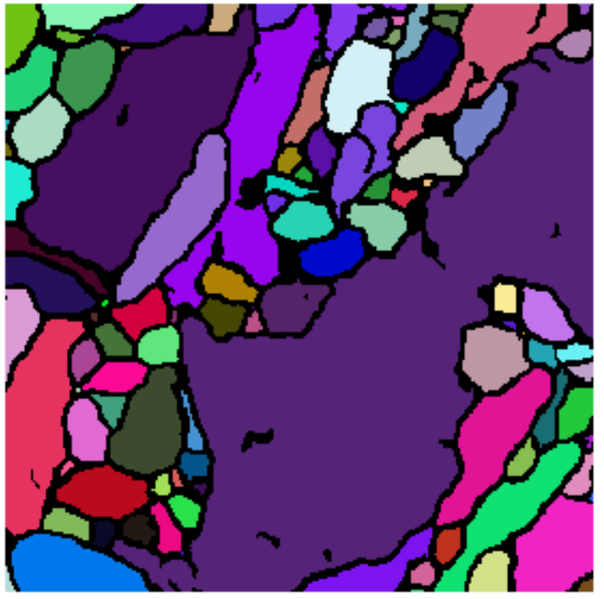
correspondingly deformed
manual labels

SHIFT, ROTATION, FLIP, ELASTIC DEFORMATION

HTTPS://WIKIDOCS.NET/148870







HTTPS://GITHUB.COM/USUYAMA/PYTORCH-UNET/BLOB/MASTER/PYTORCH_UNET.PY

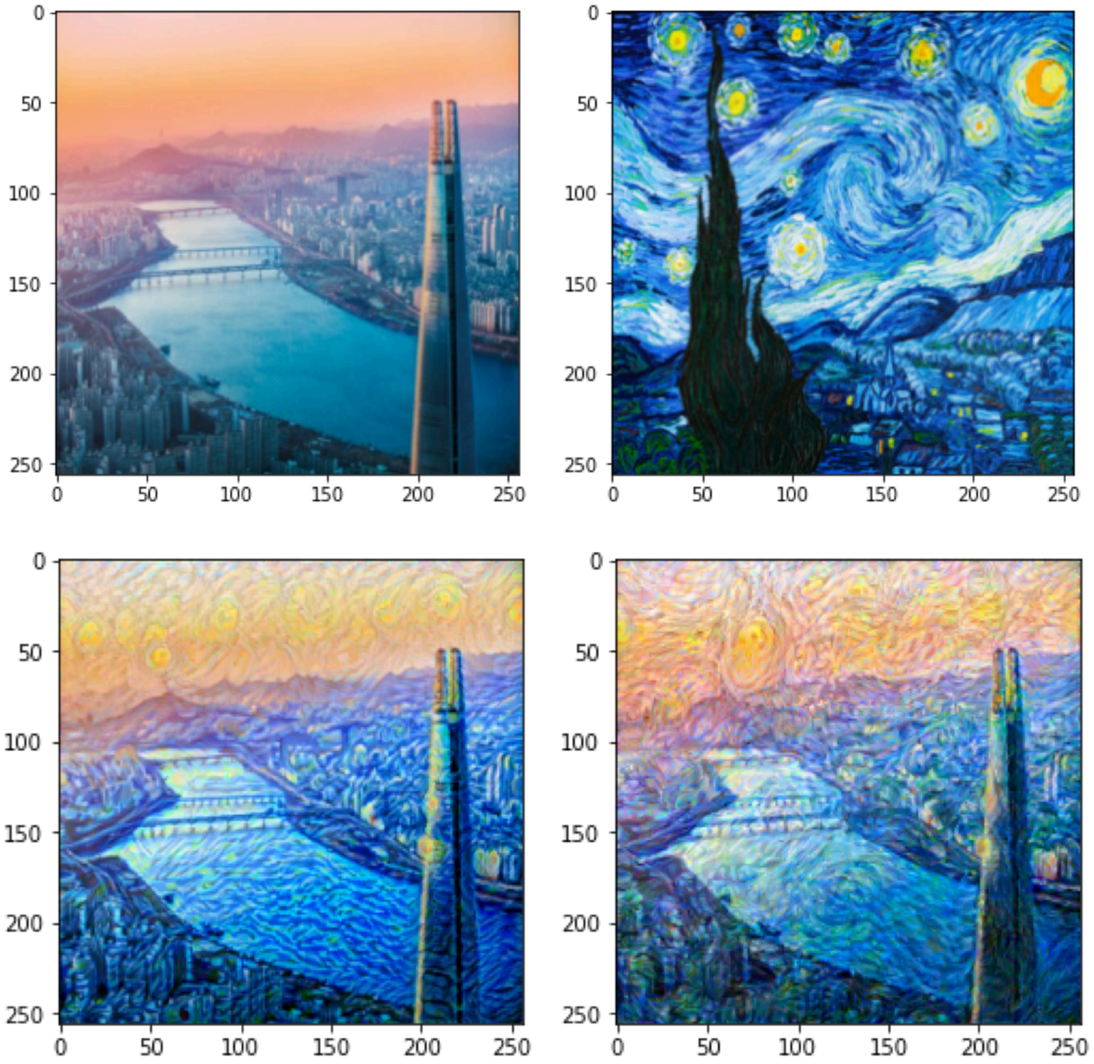
```python
def double_conv(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_channels, out_channels, 3, padding=1),
        nn.ReLU(inplace=True)
    )

class UNet(nn.Module):

    def __init__(self, n_class):
        super().__init__()

        self.dconv_down1 = double_conv(3, 64)
        self.dconv_down2 = double_conv(64, 128)
        self.dconv_down3 = double_conv(128, 256)
        self.dconv_down4 = double_conv(256, 512)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)

        self.dconv_up3 = double_conv(256 + 512, 256)
        self.dconv_up2 = double_conv(128 + 256, 128)
        self.dconv_up1 = double_conv(128 + 64, 64)

        self.conv_last = nn.Conv2d(64, n_class, 1)
```

```python
def forward(self, x):
    conv1 = self.dconv_down1(x)
    x = self.maxpool(conv1)

    conv2 = self.dconv_down2(x)
    x = self.maxpool(conv2)

    conv3 = self.dconv_down3(x)
    x = self.maxpool(conv3)

    x = self.dconv_down4(x)

    x = self.upsample(x)
    x = torch.cat([x, conv3], dim=1)

    x = self.dconv_up3(x)
    x = self.upsample(x)
    x = torch.cat([x, conv2], dim=1)

    x = self.dconv_up2(x)
    x = self.upsample(x)
    x = torch.cat([x, conv1], dim=1)

    x = self.dconv_up1(x)

    out = self.conv_last(x)

    return out
```

생성인공지능 원리

HTTPS://WWW.CV-FOUNDATION.ORG/OPENACCESS/CONTENT_CVPR_2016/PAPERS/GATYS_IMAGE_STYLE_TRANSFER_CVPR_2016_PAPER.PDF

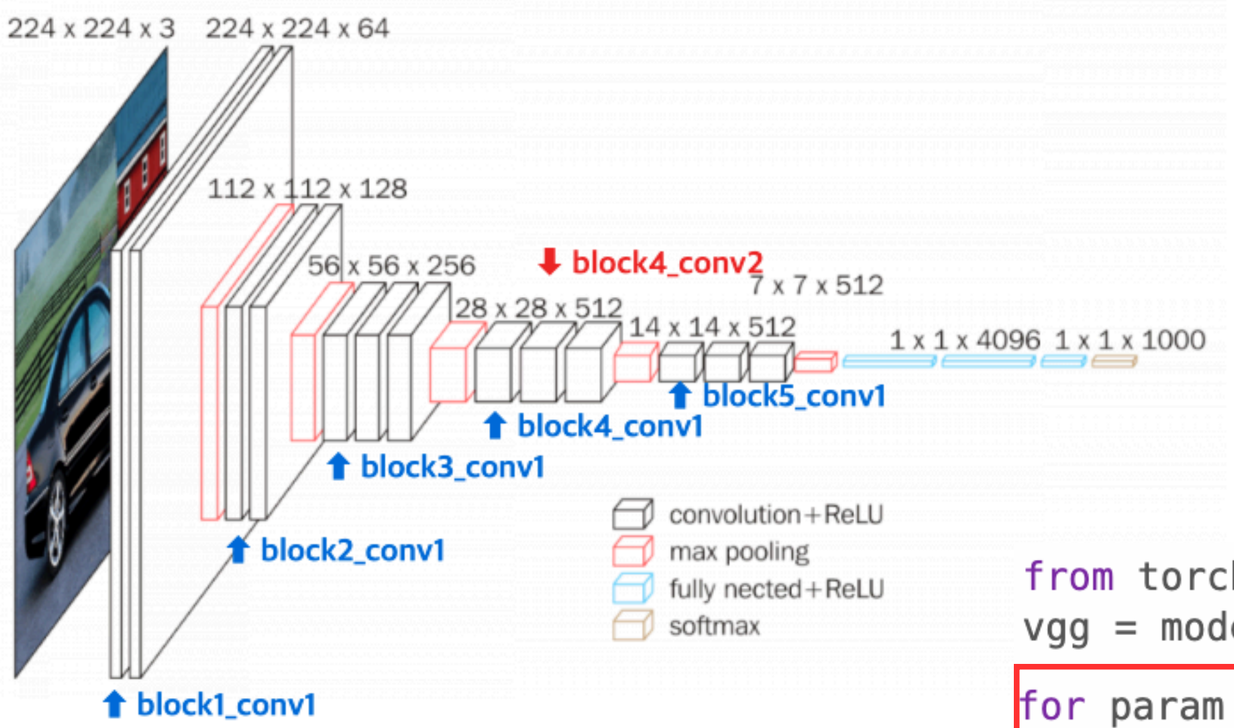
Style transfer란?



DEFINITION OF STYLE?

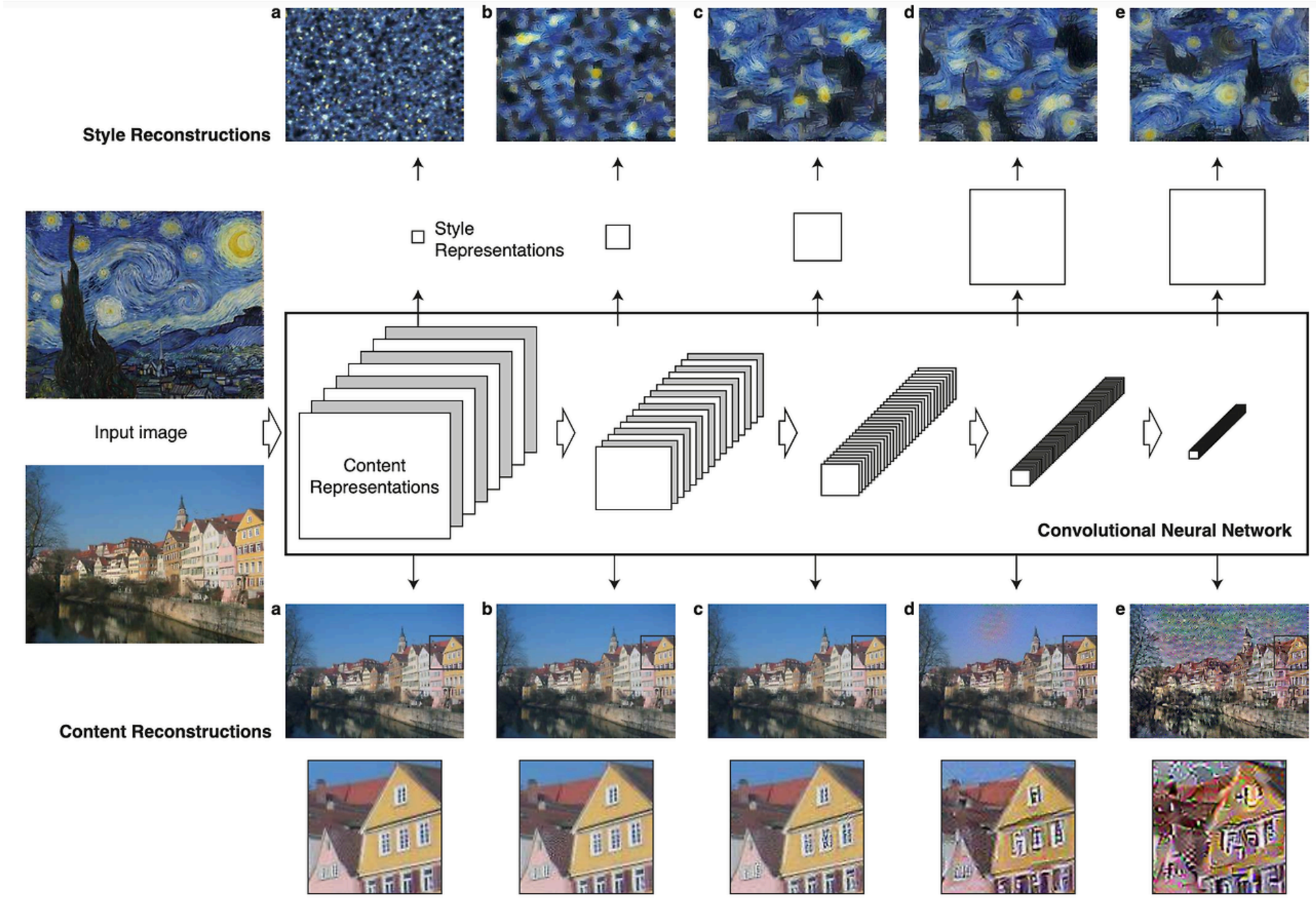
스타일이란 무엇인가?
모양? 색? 패턴?

EXTRACT FEATURES



VGG Neural Network Architecture

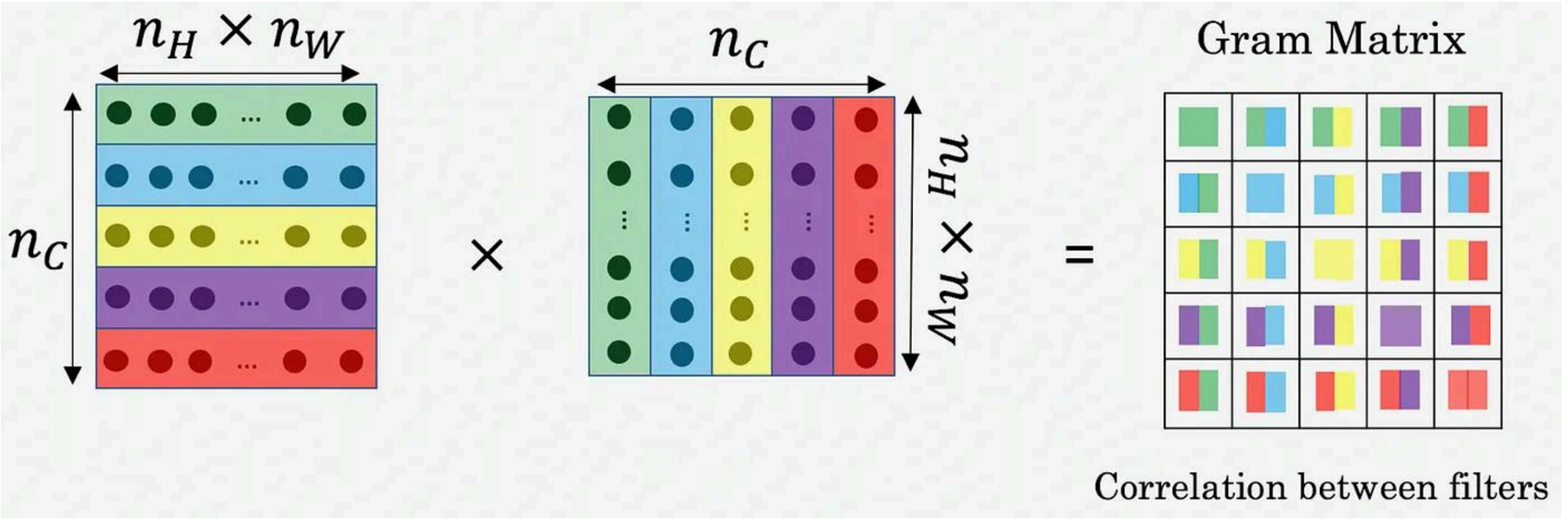
```
from torchvision import models
vgg = models.vgg19(pretrained=True).features

for param in vgg.parameters():
    param.requires_grad_(False)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

vgg.to(device)
```


a
b
c
d
e
Style Reconstructions
Style Representations
Input image
Content Representations
Convolutional Neural Network
a
b
c
d
e
Content Reconstructions

Gram Matrix

In [0]:
```
def grammatrix(tensor):
    n, d, h, w = tensor.size()
    tensor = tensor.view(d, h * w)
    grammat = torch.mm(tensor, tensor.t())
    return grammat
```

In [0]:
```
style_gram_matrix = {layer: grammatrix(style_features[layer]) for layer in style_features}
```



$$E_L = \sum \left(G^L - A^L\right)^2$$

$$\mathcal{L}_{total} = \alpha \mathcal{L}_{content} + \beta \mathcal{L}_{style}$$

$$G^L_{ij} = \sum_k F^L_{ik} F^L_{jk}.$$

$$\mathcal{L}_{content} = \sum \left(F^l - P^l\right)^2$$

$$\mathcal{L}_{style} = \sum_l w_l E_l$$

$$\vec{x} := \vec{x} - \lambda \frac{\partial \mathcal{L}_{total}}{\partial \vec{x}}$$

스타일!

모양!

이미지를 학습!!

Gradient descent

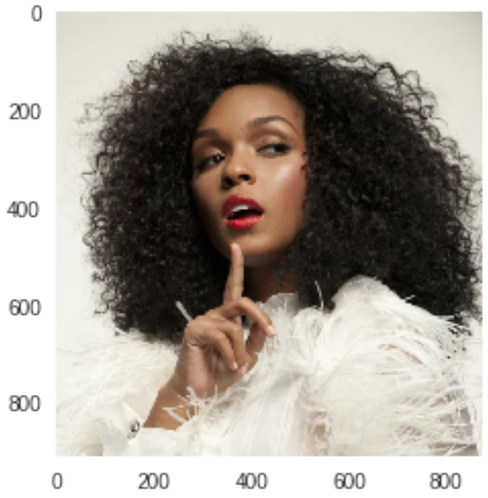
feature maps

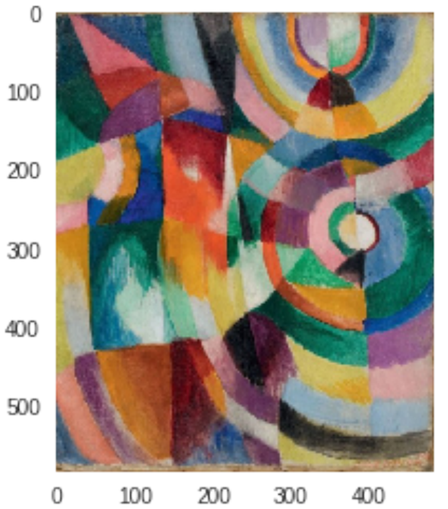
conv5_1 conv4_1 pool4 conv3_1 pool3 conv2_1 pool2 conv1_1 pool1 input

512 512 256 128 64

이미지 불러오기

```
from PIL import Image
content_img = Image.open('janelle.png')
style_img = Image.open('delaunay.jpg')
```





전처리

```
from torchvision import transforms
img_transform = transforms.Compose([transforms.Resize((400, 400)),
                transforms.ToTensor(),
                transforms.Normalize((0.485,0.456, 0.406),(0.229,0.224, 0.225))])

content = img_transform(content_img)[:3,:,:].unsqueeze(0).to(device)
style = img_transform(style_img)[:3,:,:].unsqueeze(0).to(device)
```

복원

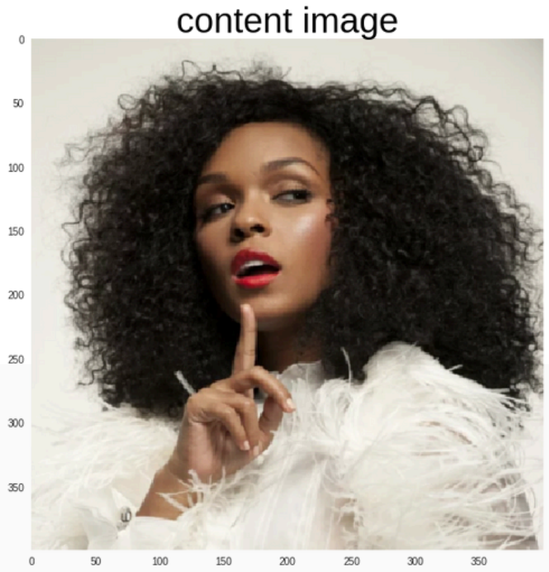
```
def im_convert(tensor):
    """ Display a tensor as an image. """

    image = tensor.to("cpu").clone().detach()
    image = image.numpy().squeeze()
    image = image.transpose(1,2,0)
    image = image * np.array((0.229, 0.224, 0.225)) + np.array((0.485, 0.456, 0.406))
    image = image.clip(0, 1)

    return image
```

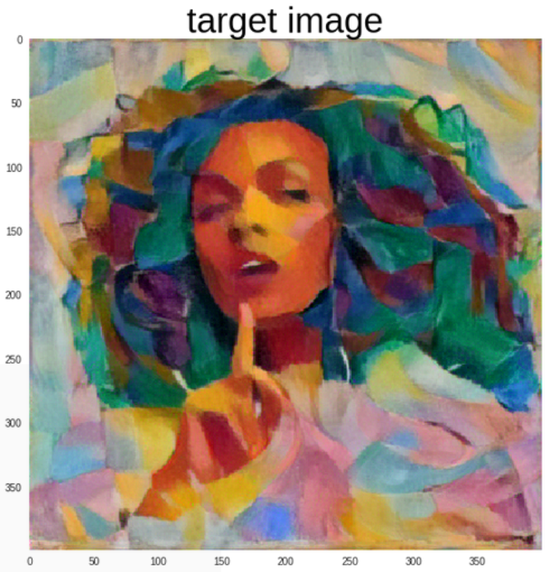
HTTPS://GITHUB.COM/SHANGETH/NEURAL-STYLE-TRANSFER/BLOB/MASTER/NEURAL_STYLE_TRANSFER.IPYNB

```python
# a general function to exrtract the features
def get_features(image, model, layers=None):
    if layers is None:
        layers = {'0': 'conv1_1',
                  '5': 'conv2_1',
                  '10': 'conv3_1',
                  '19': 'conv4_1',
                  '21': 'conv4_2',
                  '28': 'conv5_1'}
    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

```python
target = content.clone().requires_grad_(True).to(device)
```

```
style_weights = {'conv1_1': 1.,
                 'conv2_1': 0.75,
                 'conv3_1': 0.2,
                 'conv4_1': 0.2,
                 'conv5_1': 0.2}

content_weight = 1  # this is alpha in the final loss
style_weight = 1e6  # this is beta in the final loss
```






```
optimizer = optim.Adam([target], lr=0.003)
steps = 2000

for ii in range(1, steps+1):
    target_features = get_features(target, vgg)
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        target_gram = grammatrix(target_feature)
        _, d, h, w = target_feature.shape
        style_gram = style_gram_matrix[layer]
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
        style_loss += layer_style_loss / (d * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
```

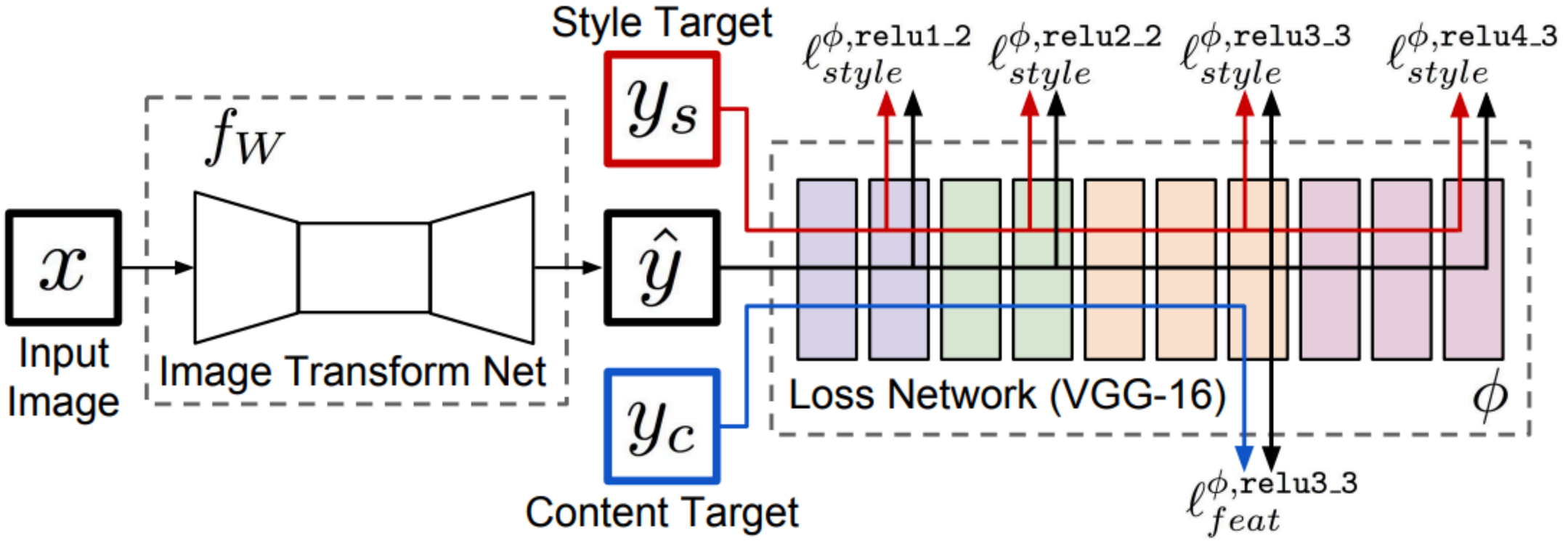


Fig. 2. System overview. We train an *image transformation network* to transform input images into output images. We use a *loss network* pretrained for image classification to define *perceptual loss functions* that measure perceptual differences in content and style between images. The loss network remains fixed during the training process.

$$\text{AdaIN}(x, y) = \sigma(y)\left(\frac{x - \mu(x)}{\sigma(x)}\right) + \mu(y)$$

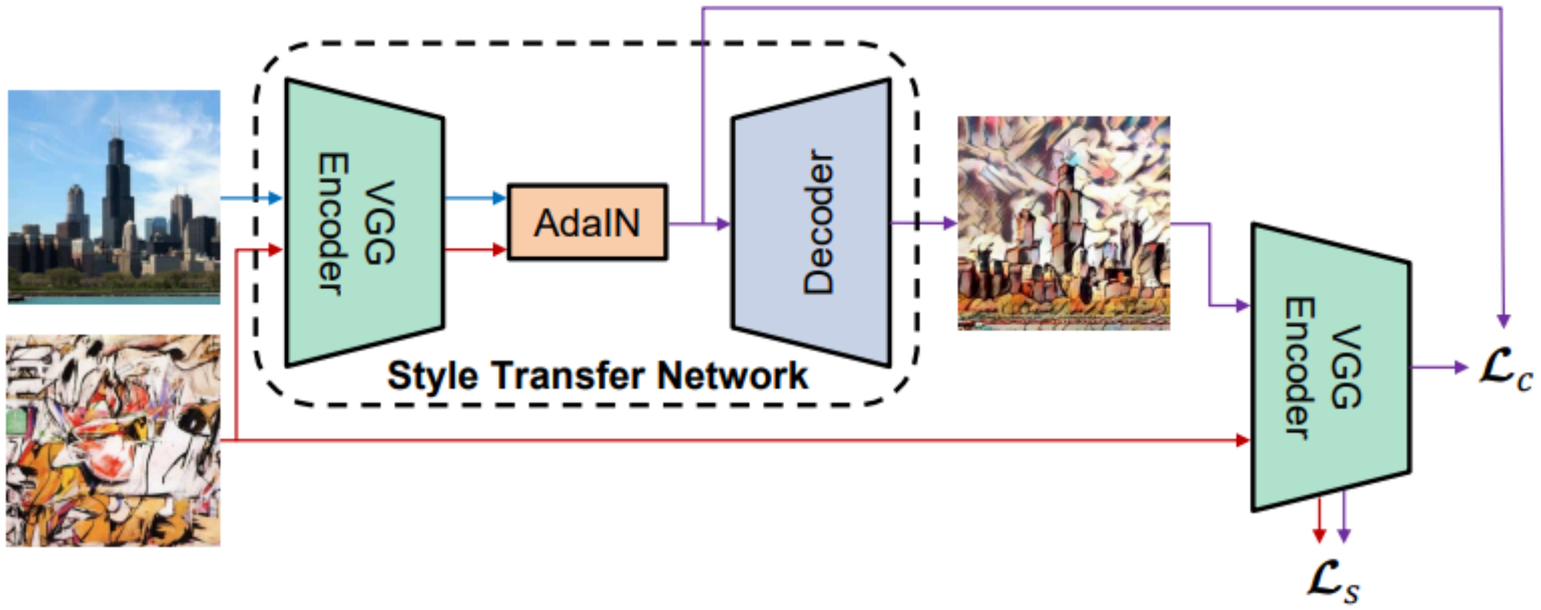
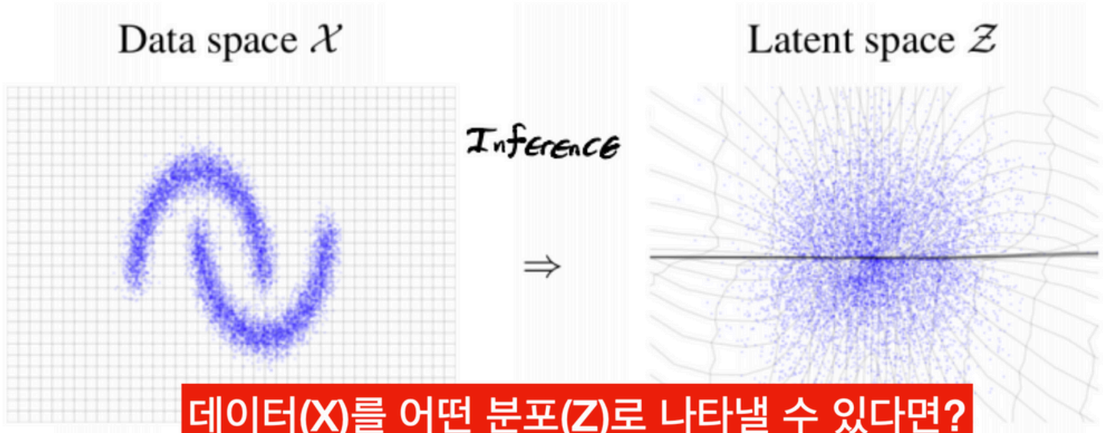


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

5-3. GENERATIVE ADVERSARIAL NETWORK

데이터?



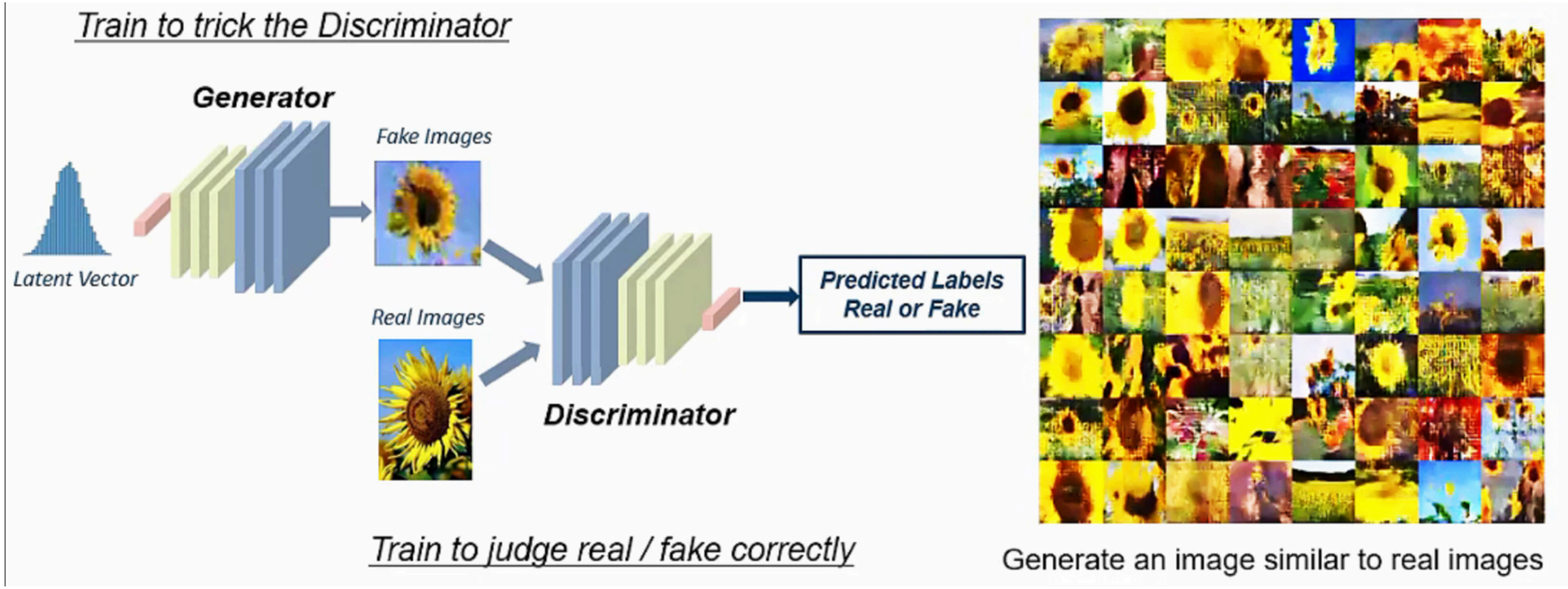
데이터(X)를 어떤 분포(Z)로 나타낼 수 있다면?

그 분포(Z)에서 데이터(X)를 만들 수 있는거 아님?

?!

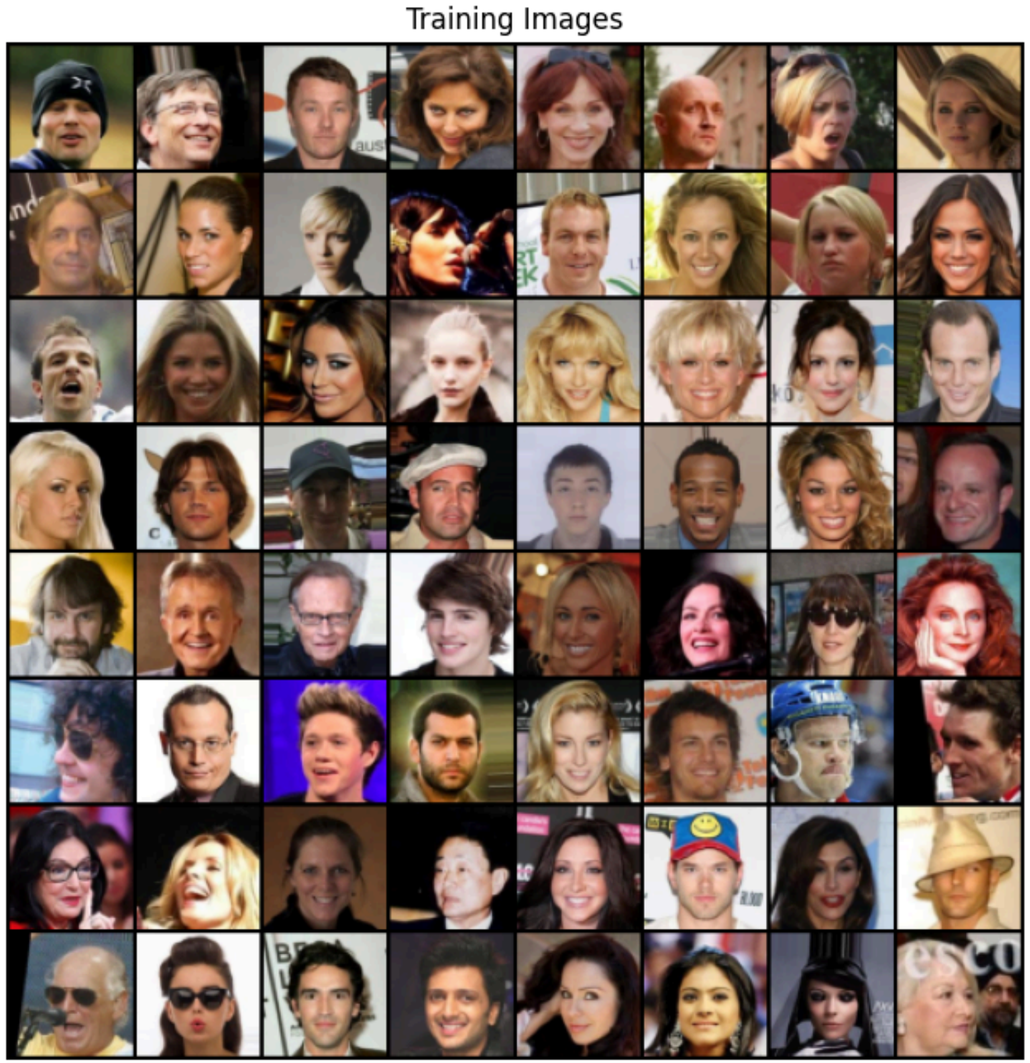
Generative Adversarial Network

Briefly elaborate on what you want to discuss.



DC-GAN

CELEBA-DATASET



HTTPS://ARXIV.ORG/PDF/1511.06434

```python
import argparse
import os
import random
import torch
import torch.nn as nn
import torch.nn.parallel
import torch.backends.cudnn as cudnn
import torch.optim as optim
import torch.utils.data
import torchvision.datasets as dset
import torchvision.transforms as transforms
import torchvision.utils as vutils
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from IPython.display import HTML

# 코드 실행결과의 동일성을 위해 무작위 시드를 설정합니다
manualSeed = 999
#manualSeed = random.randint(1, 10000) # 만일
print("Random Seed: ", manualSeed)
random.seed(manualSeed)
torch.manual_seed(manualSeed)
```

```python
# 데이터셋의 경로
dataroot = "data/celeba"

# dataloader에서 사용할 쓰레드 수
workers = 2

# 배치 크기
batch_size = 128

# 이미지의 크기입니다. 모든 이미지를 변환하여 64로 크기가 통일됩니다.
image_size = 64

# 이미지의 채널 수로, RGB 이미지이기 때문에 3으로 설정합니다.
nc = 3

# 잠재공간 벡터의 크기 (예. 생성자의 입력값 크기)
nz = 100

# 생성자를 통과하는 특징 데이터들의 채널 크기
ngf = 64

# 구분자를 통과하는 특징 데이터들의 채널 크기
ndf = 64

# 학습할 에폭 수
num_epochs = 5

# 옵티마이저의 학습률
lr = 0.0002

# Adam 옵티마이저의 beta1 하이퍼파라미터
beta1 = 0.5

# 사용가능한 gpu 번호. CPU를 사용해야 하는경우 0으로 설정하세요
ngpu = 1
```

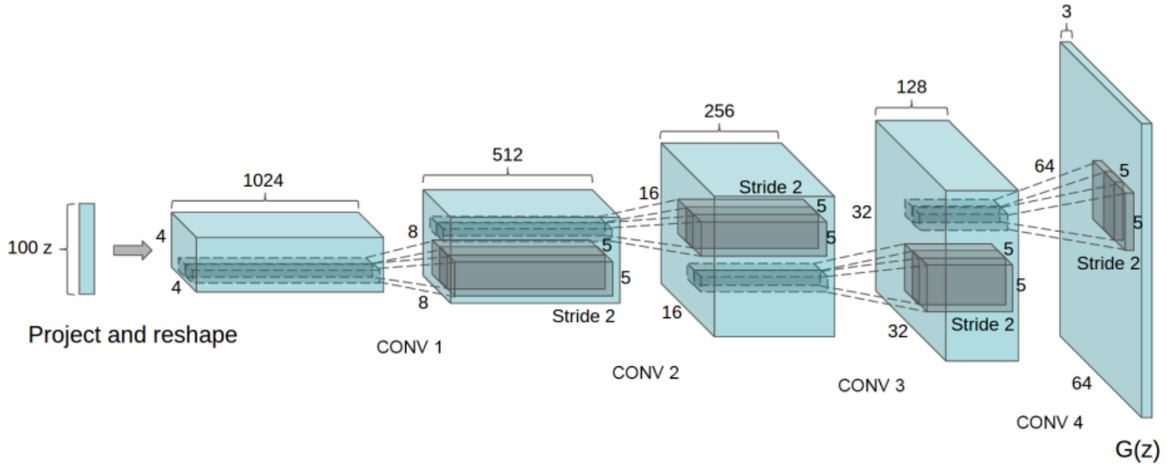
HTTPS://TUTORIALS.PYTORCH.KR/BEGINNER/DCGAN_FACES_TUTORIAL.HTML

```
# 우리가 설정한 대로 이미지 데이터셋을 불러와 봅시다
# 먼저 데이터셋을 만듭니다
dataset = dset.ImageFolder(root=dataroot,
                           transform=transforms.Compose([
                               transforms.Resize(image_size),
                               transforms.CenterCrop(image_size),
                               transforms.ToTensor(),
                               transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
                           ]))
# dataloader를 정의해봅시다
dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                         shuffle=True, num_workers=workers)

# GPU 사용여부를 결정해 줍니다
device = torch.device("cuda:0" if (torch.cuda.is_available() and ngpu > 0) else "cpu")

# 학습 데이터들 중 몇가지 이미지들을 화면에 띄워봅시다
real_batch = next(iter(dataloader))
plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Training Images")
plt.imshow(np.transpose(vutils.make_grid(real_batch[0].to(device)[:64], padding=2, normalize=True).cpu(),(1,2,0)))
```

DCGAN 논문에서는, 평균이 0(MEAN=0)이고 분산이 0.02(STDEV=0.02)인 정규분포을 시용해, 구분자와 생성자 모두 무작위 초기화를 진행하는 것이 좋다고 합니다.

```python
# ``netG`` 와 ``netD`` 에 적용시킬 커스텀 가중치 초기화 함수
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

```python
class Generator(nn.Module):
    def __init__(self, ngpu):
        super(Generator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # 입력데이터 Z가 가장 처음 통과하는 전치 합성곱 계층입니다.
            nn.ConvTranspose2d( nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # 위의 계층을 통과한 데이터의 크기. ``(ngf*8) x 4 x 4``
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # 위의 계층을 통과한 데이터의 크기. ``(ngf*4) x 8 x 8``
            nn.ConvTranspose2d( ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            # 위의 계층을 통과한 데이터의 크기. ``(ngf*2) x 16 x 16``
            nn.ConvTranspose2d( ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            # 위의 계층을 통과한 데이터의 크기. ``(ngf) x 32 x 32``
            nn.ConvTranspose2d( ngf, nc, 4, 2, 1, bias=False),
            nn.Tanh()
            # 위의 계층을 통과한 데이터의 크기. ``(nc) x 64 x 64``
        )

    def forward(self, input):
        return self.main(input)
```



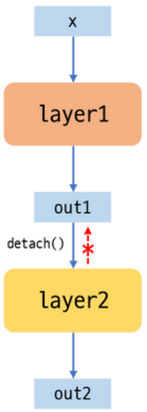

```python
# 생성자를 만듭니다
netG = Generator(ngpu).to(device)

# 필요한 경우 multi-GPU를 설정 해주세요
if (device.type == 'cuda') and (ngpu > 1):
    netG = nn.DataParallel(netG, list(range(ngpu)))

# 모든 가중치의 평균을 0( ``mean=0`` ), 분산을 0.02( ``stdev=0.02`` )로 초기화하기 위해
# ``weight_init`` 함수를 적용시킵니다
netG.apply(weights_init)

# 모델의 구조를 출력합니다
print(netG)
```

```python
class Discriminator(nn.Module):
    def __init__(self, ngpu):
        super(Discriminator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # 입력 데이터의 크기는 ``(nc) x 64 x 64`` 입니다
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 위의 계층을 통과한 데이터의 크기. ``(ndf) x 32 x 32``
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # 위의 계층을 통과한 데이터의 크기. ``(ndf*2) x 16 x 16``
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # 위의 계층을 통과한 데이터의 크기. ``(ndf*4) x 8 x 8``
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # 위의 계층을 통과한 데이터의 크기. ``(ndf*8) x 4 x 4``
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input)
```

```python
# 구분자를 만듭니다
netD = Discriminator(ngpu).to(device)

# 필요한 경우 multi-GPU를 설정 해주세요
if (device.type == 'cuda') and (ngpu > 1):
    netD = nn.DataParallel(netD, list(range(ngpu)))

# 모든 가중치의 평균을 0( ``mean=0`` ), 분산을 0.02( ``stdev=0.02`` )로 초기화하기 위해
# ``weight_init`` 함수를 적용시킵니다
netD.apply(weights_init)

# 모델의 구조를 출력합니다
print(netD)
```

```
# ``BCELoss`` 함수의 인스턴스를 초기화합니다
criterion = nn.BCELoss()

# 생성자의 학습상태를 확인할 잠재 공간 벡터를 생성합니다
fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# 학습에 사용되는 참/거짓의 라벨을 정합니다
real_label = 1.
fake_label = 0.

# G와 D에서 사용할 Adam옵티마이저를 생성합니다
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))
```

```python
# 학습 과정

# 학습상태를 체크하기 위해 손실값들을 저장합니다
img_list = []
G_losses = []
D_losses = []
iters = 0

print("Starting Training Loop...")
# 에폭(epoch) 반복
for epoch in range(num_epochs):
    # 한 에폭 내에서 배치 반복
    for i, data in enumerate(dataloader, 0):

        ############################
        # (1) D 신경망을 업데이트 합니다: log(D(x)) + log(1 - D(G(z)))를 최대화 합니다
        ###########################
        ## 진짜 데이터들로 학습을 합니다
        netD.zero_grad()
        # 배치들의 사이즈나 사용할 디바이스에 맞게 조정합니다
        real_cpu = data[0].to(device)
        b_size = real_cpu.size(0)
        label = torch.full((b_size,), real_label,
                           dtype=torch.float, device=device)
        # 진짜 데이터들로 이루어진 배치를 D에 통과시킵니다
        output = netD(real_cpu).view(-1)
        # 손실값을 구합니다
        errD_real = criterion(output, label)
        # 역전파의 과정에서 변화도를 계산합니다
        errD_real.backward()
        D_x = output.mean().item()

        ## 가짜 데이터들로 학습을 합니다
        # 생성자에 사용할 잠재공간 벡터를 생성합니다
        noise = torch.randn(b_size, nz, 1, 1, device=device)
        # G를 이용해 가짜 이미지를 생성합니다
        fake = netG(noise)
        label.fill_(fake_label)
        # D를 이용해 데이터의 진위를 판별합니다
        output = netD(fake.detach()).view(-1)
        # D의 손실값을 계산합니다
        errD_fake = criterion(output, label)
        # 역전파를 통해 변화도를 계산합니다. 이때 앞서 구한 변화도에 더합니다(accumulate)
        errD_fake.backward()
        D_G_z1 = output.mean().item()
        # 가짜 이미지와 진짜 이미지 모두에서 구한 손실값들을 더합니다
        # 이때 errD는 역전파에서 사용되지 않고, 이후 학습 상태를 리포팅(reporting)할 때 사용합니다
        errD = errD_real + errD_fake
        # D를 업데이트 합니다
        optimizerD.step()

        ############################
        # (2) G 신경망을 업데이트 합니다: log(D(G(z)))를 최대화 합니다
        ###########################
        netG.zero_grad()
        label.fill_(real_label)  # 생성자의 손실값을 구하기 위해 진짜 라벨을 이용할 겁니다
        # 우리는 방금 D를 업데이트했기 때문에, D에 다시 가짜 데이터를 통과시킵니다.
        # 이때 G는 업데이트되지 않았지만, D가 업데이트 되었기 때문에 앞선 손실값가 다른 값이 나오게 됩니다
        output = netD(fake).view(-1)
        # G의 손실값을 구합니다
        errG = criterion(output, label)
        # G의 변화도를 계산합니다
        errG.backward()
        D_G_z2 = output.mean().item()
        # G를 업데이트 합니다
        optimizerG.step()

        # 훈련 상태를 출력합니다
        if i % 50 == 0:
            print('[%d/%d][%d/%d]\tLoss_D: %.4f\tLoss_G: %.4f\tD(x): %.4f\tD(G(z)): %.4f / %.4f'
                  % (epoch, num_epochs, i, len(dataloader),
                     errD.item(), errG.item(), D_x, D_G_z1, D_G_z2))

        # 이후 그래프를 그리기 위해 손실값들을 저장해둡니다
        G_losses.append(errG.item())
        D_losses.append(errD.item())

        # fixed_noise를 통과시킨 G의 출력값을 저장해둡니다
        if (iters % 500 == 0) or ((epoch == num_epochs-1) and (i == len(dataloader)-1)):
            with torch.no_grad():
                fake = netG(fixed_noise).detach().cpu()
            img_list.append(vutils.make_grid(fake, padding=2, normalize=True))

        iters += 1
```



https://bo-10000.tistory.com/

DETACH한 이유?

require_grad=False인 fake의 복제품을 return fake를 재활용하려고.

```python
for i in range(EPOCH):
    for j in range(k):

        for batch_idx, (samples, _) in enumerate(data_loader):

            z = torch.randn((samples.shape[0], latent_dim)).cuda()
            X = samples.cuda()

            optimizer_D.zero_grad()
            real_result_x = discriminator(X)

            made_x = generator(z)
            fake_result_x = discriminator(made_x)

            #D term maximize
            loss_D_front = -1 * torch.log(real_result_x)
            loss_D_end = -1 * torch.log(1. -fake_result_x)
            loss_D = (loss_D_front + loss_D_end).mean()

            loss_D.backward(retain_graph=True)
            optimizer_D.step()

            ########G 학습
            optimizer_G.zero_grad()
            #G term minimize -> - 붙여서 maximize 하기(= -1 * torch.log(fake_result_x) minimize
            loss_G = -1 * torch.log(fake_result_x).mean()
            #loss_G = torch.log(1. - fake_result_x).mean()
            loss_G.backward(retain_graph=True)
            optimizer_G.step()

            done = i * len(data_loader) + batch_idx
```

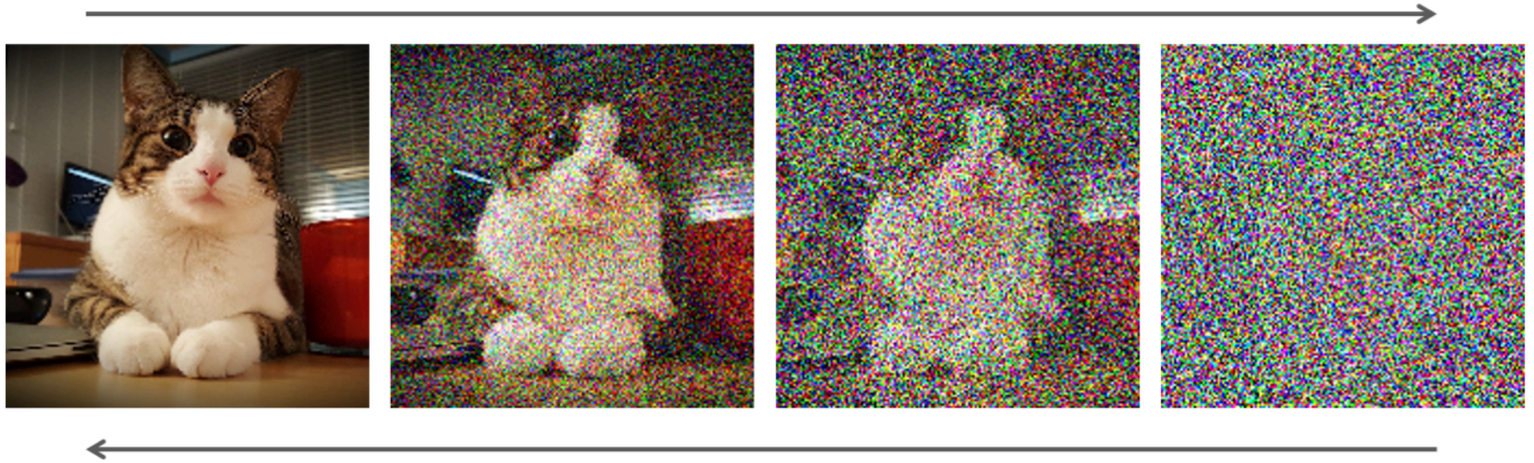
```python
batch_size = 128
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
        nn.Linear(1 * 28 * 28, 256),
        nn.ReLU(),
        nn.Linear(256, 1),
        nn.Sigmoid())

    def forward(self, img):  #한 장의 이미지가 들어왔을 때 fla
        flattened = img.view(batch_size, -1)  #(32, )로
        output = self.model(flattened)

        return output

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
        nn.Linear(100, 256),
        nn.ReLU(),
        nn.Linear(256, 512),
        nn.ReLU(),
        nn.Linear(512, 1* 28* 28),
        nn.Sigmoid())

    def forward(self, input): #input : 100차원 noise
        img = self.model(input)
        img = img.view(batch_size, 1, 28, 28)
        return img
```

비슷한 방법으로
retain_graph = True 를 쓸 수도있다

torch를 쓸때 자주 보이는 옵션

HTTPS://NOTOU10.GITHUB.IO/CAPSTONE/2021/02/05/CAPSTONE_4.HTML

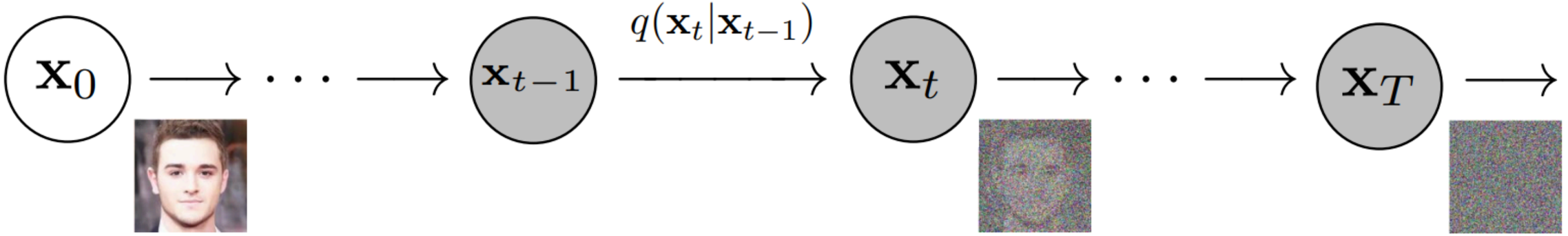
Figure 10: More face generations from our Face DCGAN.

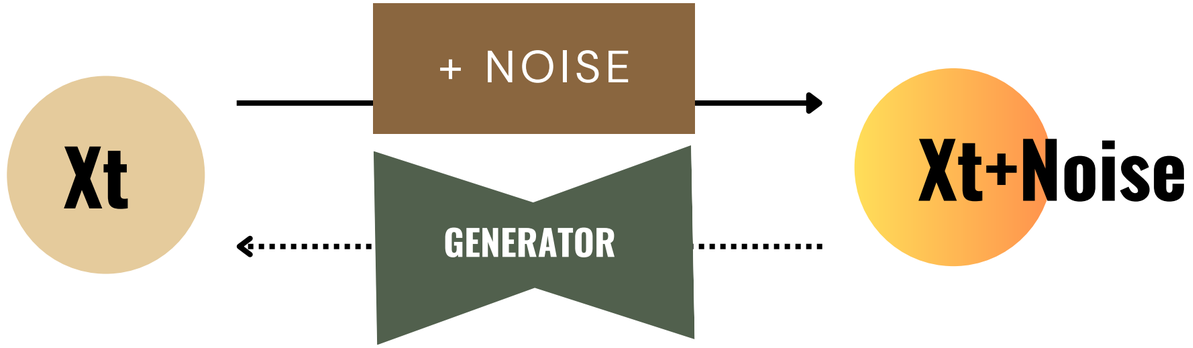


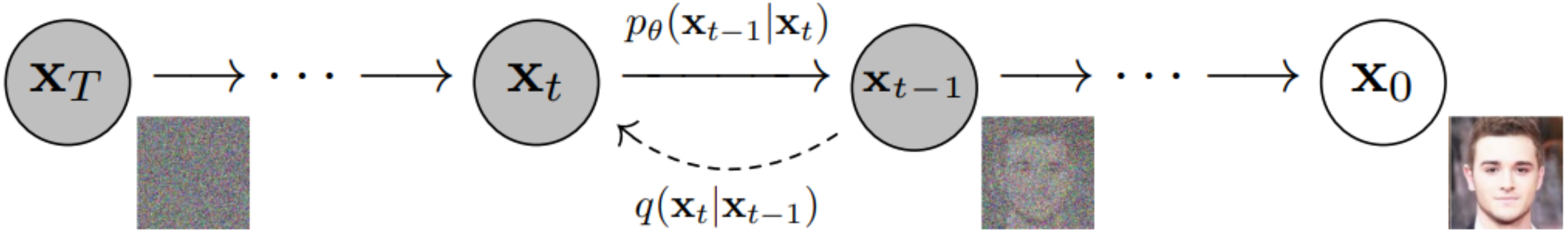
디퓨젼 모델의 기본 가정

변화가 매우 작을때, 이를 정규분포라 가정할 수 있음.

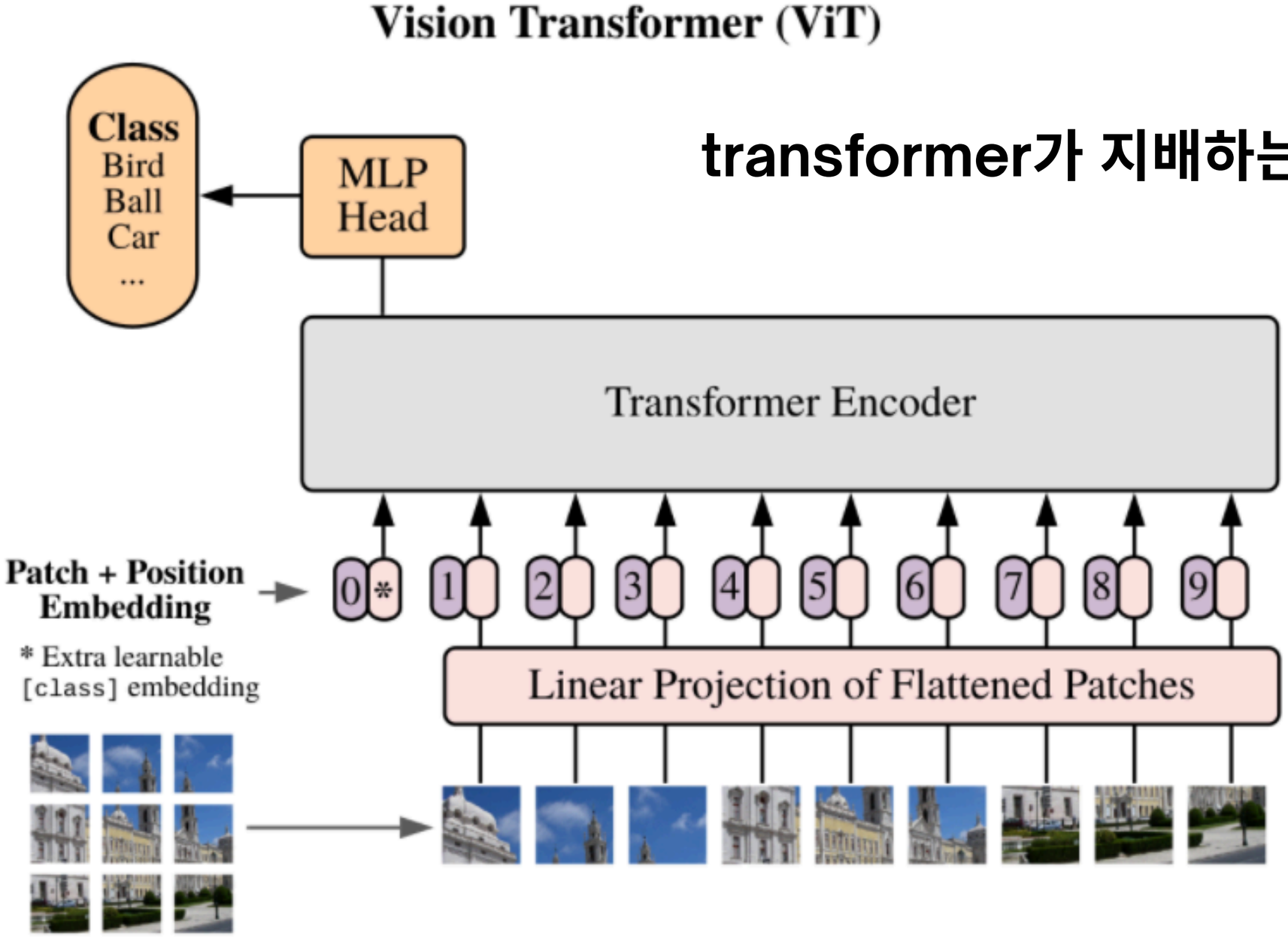
FORWARD DIFFUSION



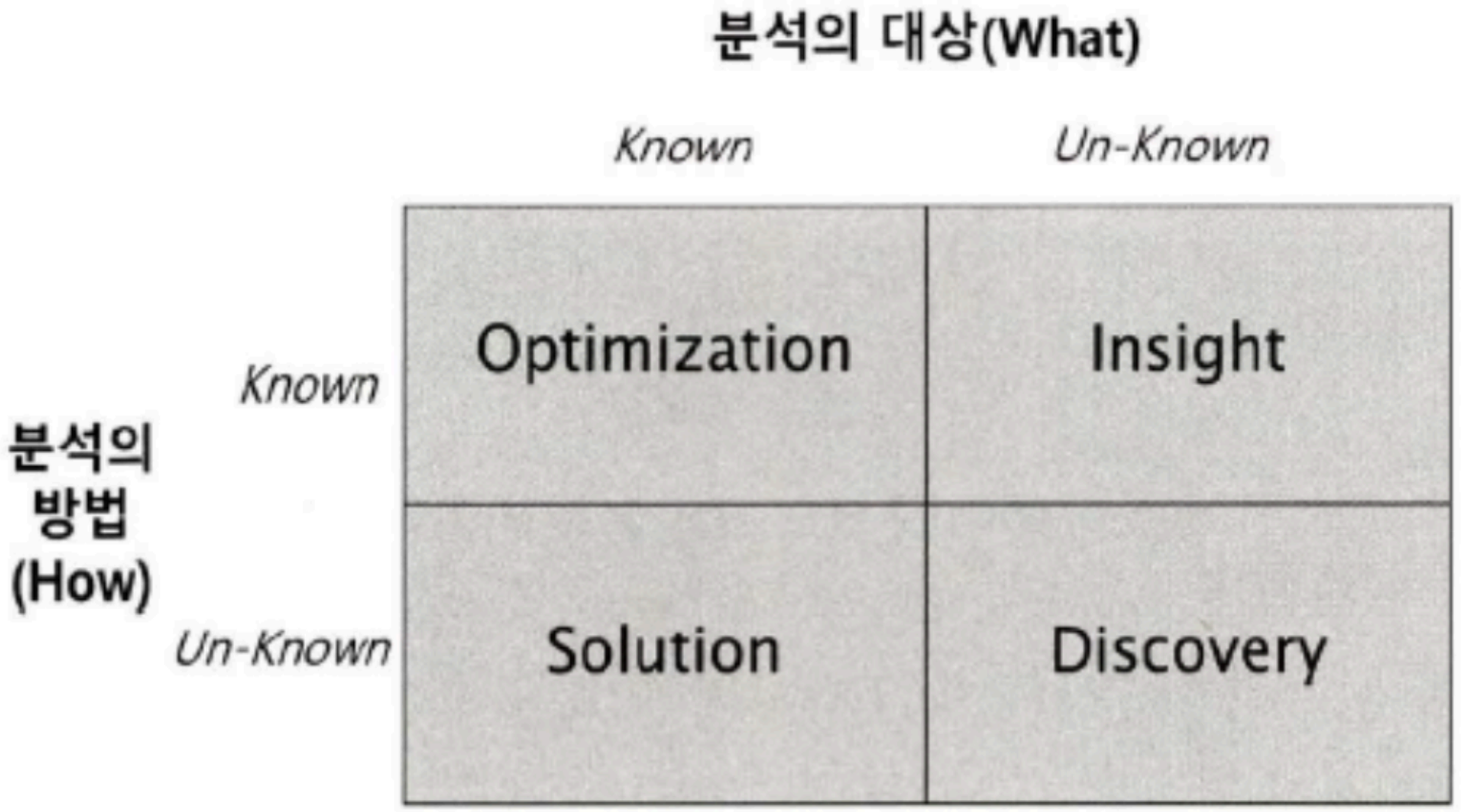




BACKWARD DIFFUSION



transformer가 지배하는 세상!


분석의 대상(What)
Known
Un-Known
분석의 방법 (How)
Known
Un-Known
Optimization
Insight
Solution
Discovery